RESEARCH WITH INKHAVEN

Grounding Your Book in Fact

Researching Fiction and Non-Fiction with Inkhaven's Research Assistant

Vladimir Ulogov

2026 · examples assume Inkhaven 1.8.30 or newer

Contents

Before You Begin

This is a book about getting the facts right. Not the invented facts — the ones you make up freely, which are yours to command — but the **borrowed** ones: the year a bridge was built, the population of a city, the distance a rider could cover in a day, the claim in your argument that a reader could look up and check. Every book leans on facts it did not invent, and a book that gets them wrong loses the reader's trust in the ones it did.

You will do this inside **Inkhaven** — a writing tool with a built-in **Research Assistant**: a quiet workspace where you ask questions, gather answers from trustworthy places, check them against each other, and keep only what survives. By the end of this book you will be able to take a manuscript from "I think that's about right" to a knowledge base where every fact you kept remembers **where it came from** — and to compose from that knowledge base directly into your writing.

Who this book is for

This guide assumes **no prior knowledge** — of research, or of Inkhaven.

You do not need to be a researcher. You do not need to know what a citation manager is, or how a database works, or the difference between a primary and a secondary source. Every idea is introduced the first time it is needed, in a marked box like this one:

> TERM **Fact**
>
> In this book, a **fact** is a specific, checkable claim you want your writing to rest on — a date, a number, a name, a cause. It is the opposite of a thing you invented. The whole craft here is telling the two apart, and grounding the first kind.

You do not need to be a novelist, either. The Research Assistant serves two kinds of author equally, and this book keeps both in view the whole way through. Where

a task looks different depending on what you write, you will see it split into two tracks:

FICTION

You are writing a novel set during the building of a Roman aqueduct. You need the engineering to feel **true** — real distances, plausible dates — without stopping the story to fact-hunt.

NON-FICTION

You are writing a history of Roman water supply. You need every claim **sourced** — a real citation behind each figure — so a reviewer can follow your evidence.

Same tool, same workflow, two destinations: the novelist wants the world to feel solid; the non-fiction author wants the argument to be defensible. Both are the same act — grounding a claim — and both are what the Research Assistant is for.

What you will be able to do

By the last page you will be able to:

- ask a question and get an answer **grounded on facts you already trust**, not a confident guess;
- pull facts from authoritative places — a structured knowledge base, real places, scholarly papers, public-domain books, the open web — each one remembering its source;
- cross-check a shaky claim against several independent sources before you commit it, and let the tool argue **against** a fact to see if it survives;
- keep your knowledge base honest over time — re-grounding old guesses, flagging facts that may have gone stale;
- and compose **out** of the corpus you built: a cited synthesis, an outline, a real bibliography — straight into the book you are writing.

What you will need

Inkhaven, installed and open on a project — a folder with a manuscript in it. That is all. Nothing in this book requires an account, a subscription, or an internet connection you have to pay for. A few features reach out to free public services

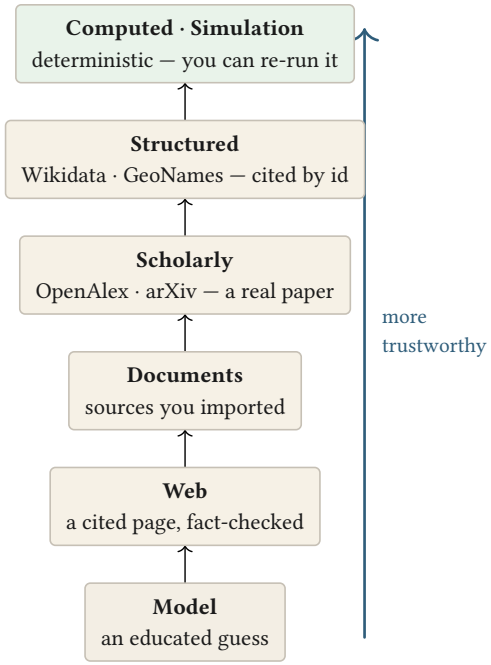
(a maps database, a scholarly index, a library of public-domain books); each one is optional, degrades gracefully when you are offline, and is clearly marked where it appears.

ON THE AI

The Research Assistant uses a language model to help you **find** and **phrase** answers — but it never decides what is true for you, and it never edits your prose. Every fact crosses a confirmation step before it is kept, and every fact records where it came from. The model is a research partner with a short leash, not an oracle. If you have no model configured, most of the workflow still works; the parts that need one say so.

The one idea to carry with you

If you remember nothing else from this introduction, remember this picture. It is the spine of the whole book:



The trust ladder — every fact you keep carries a rung, recorded as its provenance.

Not all facts are equally trustworthy, and the Research Assistant never pretends otherwise. A number you **computed** is firmer than one a structured database gives you; that is firmer than a scholarly paper; that is firmer than a web page; and all of them are firmer than a model's unaided guess. Every fact you keep is stamped with its rung on this ladder — its **provenance** — so that later, you (or a reader, or a reviewer) can see at a glance how much weight it can bear.

Everything else in this book is a way of climbing that ladder: getting a claim from a guess at the bottom to something cited and checked near the top.

How to read this book

Read it front to back the first time. Part I builds the foundation — why grounding matters, the workspace you will live in, and your very first grounded fact — and everything after it assumes those three chapters. Later parts are more independent: once you have the basics, you can jump to the source you care about (real places, scholarly papers, the web) or the technique you need (cross-checking, composing a bibliography).

Commands you type look like `/fact` or `/geonames`. Keys you press look like `Ctrl+S`. When a chord opens something on screen, the book tells you what you would see and why it matters — but it shows you the **shape** of the work with diagrams rather than screenshots, because screenshots age badly and the ideas underneath them do not.

Turn the page, and let us talk about why any of this matters.

WHAT YOU LEARNED

- A **fact** here is a borrowed, checkable claim — the opposite of what you invented — and grounding it is the whole craft.
- The book serves **fiction and non-fiction** authors equally, and assumes no prior knowledge of research or Inkhaven.
- Facts differ in trustworthiness; the **trust ladder** ranks them, and every kept fact records its rung (its **provenance**).
- The AI helps you find and phrase, but never decides truth and never edits your prose — every fact crosses a confirmation step.

PART I

Grounding Your Book

CHAPTER 1

1

Why Ground Your Book

A reader gives you enormous latitude. They will follow you into an invented world, accept a magic system, believe in characters who never lived, forgive a coincidence that would never happen. What they will not forgive is being made to feel foolish — and nothing does that faster than a fact they **know** to be wrong.

The moment a reader catches you saying the aqueduct carried water uphill, or that a medieval army marched two hundred miles in a day, or that a chemical does something it cannot do, a small door closes. Not because the error is large, but because it reveals that you did not check — and if you did not check **that**, what else did you not check? The invented facts and the borrowed facts sit side by side on the page, and the reader cannot always tell which is which. So they trust all of them,

or none. The borrowed facts are load-bearing precisely because a reader **can** look them up.

Two kinds of claim

Every sentence you write that makes a claim about the world is one of two kinds.

Some claims are **yours**. The name of your protagonist, the shape of your invented coastline, the rule that your magic costs the caster a memory — these you command absolutely. There is no external truth to get wrong. We will come back to these in a later chapter, because they deserve protection **from** fact-checking, not by it.

Other claims are **borrowed**. They rest on a world that exists independently of your book — history, geography, science, the record of what actually happened. You did not invent the speed of a horse or the year Rome fell, and a reader can check them. These are the claims this book is about.

> TERM **Grounding**
>
> **Grounding** a claim means connecting it to something outside your own head that supports it — a source, a computation, a cross-checked agreement between independent references. A grounded claim can answer the question "how do you know?" An ungrounded one can only answer "it felt right."

The first discipline of research, then, is simply **telling the two apart**. A surprising amount of trouble comes from treating a borrowed claim as if it were yours — writing "the journey took three days" because it felt right, when the distance and the terrain would have made it eight. The Research Assistant cannot tell you which claims to borrow. But once you know a claim is borrowed, it gives you everything you need to ground it.

FICTION

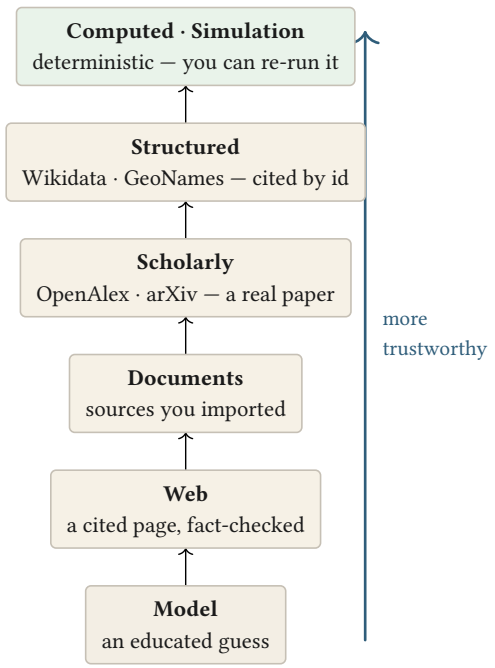
In a novel, grounding is **invisible craft**. The reader never sees your sources; they feel the result — a world that holds together, that never trips them. You ground so the seams don't show.

NON-FICTION

In non-fiction, grounding is **the argument itself**. The reader is meant to see your sources; a claim without one is an assertion, and assertions don't persuade. You ground so the case can be followed.

Not all grounding is equal

Here is the idea from the introduction again, because everything hangs on it. When you ground a claim, **what you ground it on matters**. A source can be more or less trustworthy, and the Research Assistant is honest about the difference at every step. It arranges the ways of knowing into a ladder:



The trust ladder — every fact you keep carries a rung, recorded as its provenance.

Read it from the bottom up.

Model — an educated guess
At the bottom sits the language model answering from memory. It is often right, frequently fluent, and occasionally confidently wrong — and it cannot tell you which. A model's unaided answer is a **starting point**, never a resting place. Useful for finding your question; never sufficient for keeping the answer.

Web — a cited page
A step up: an answer drawn from actual web pages, with the pages named. Now there is something to check. The Assistant fact-checks a web-grounded claim before it lets you keep it, and records the page it came from.

Documents — sources you imported
Higher still: your own trusted materials — a PDF, a research folder, a book — brought into the corpus so answers can be drawn from **them** specifically, and cited back to the page.

Scholarly — a real paper
A named work in the scholarly record (a journal article, a preprint), with an identifier a reviewer can resolve. The claim now rests on the literature.

Structured — cited by id
A structured knowledge base — geographic gazetteers, curated fact databases — where a claim comes as a discrete, identified datum rather than a paragraph of prose. Not "an article that mentions Rome," but "Rome, id Q220, founded –753."

Computed · Simulation — deterministic
At the top, facts you did not look up but **derived** — a distance from coordinates, a travel time from a speed, a fact your own world-simulation produced. These are the firmest of all, because anyone can re-run the computation and get the same answer.

> **WHY A LADDER AND NOT A WALL**
>
> The ladder does not mean "only ever use the top rung." A novelist grounding the feel of a city may be perfectly served by a web page; a historian needs the scholarly rung or higher. The point is not to forbid the low rungs — it is to **know which rung you are standing on**, and to record it, so nothing pretends to be firmer than it is.

Where your facts live

Grounded claims have to go somewhere. In Inkhaven they live in a small set of places, kept separate on purpose:

- **Facts** — the book you trust. A claim only lands here after it has been grounded and you have confirmed it. This is your ground truth.
- **Notes** — the book you are still unsure about. A promising claim that has not earned the Facts book yet; a lead to chase later.
- **Sources** — your bibliography. The citable works behind your facts — papers, books, imported references — accumulated as you research, ready to become a real reference list.

> TERM **Corpus**
>
> Your **corpus** is the whole body of research material you have gathered for a project — the Facts, the Notes, the Sources, and the imported documents behind them. It grows as you work and, crucially, you can search and compose from it. It is the difference between "I read a lot for this book" and "I can find and cite exactly what I read."

The separation matters. Keeping speculative Notes out of your trusted Facts is what lets you **rely** on the Facts — when you pull a fact into your manuscript, you are pulling from a book you vetted, not from a pile where the checked and the unchecked are mixed together.

Provenance: the fact that remembers

The single most important habit this book will build is small and almost invisible: **every fact you keep remembers where it came from.**

TERM **Provenance**

Provenance is the recorded origin of a fact — which rung of the trust ladder it came from, and the specific source (a page id, a paper, a URL, a computation). It travels with the fact silently, and you can call it up any time to ask a fact "how do you know?"

Provenance is what turns a knowledge base from a heap into an instrument. Six months from now, when a copy-editor queries a date, you will not have to reconstruct where you found it — the fact will tell you. When you assemble a bibliography, it will build itself from the provenance you accumulated. And when a fact is only a model's guess, its provenance says so plainly, so you know it still needs work.

You will rarely type the word "provenance" again. But it is running underneath every command in this book, and it is the reason the Research Assistant can be trusted to be honest about what it knows.

WHAT YOU LEARNED

- Borrowed (checkable) claims are load-bearing: one caught error makes a reader doubt the rest, invented and borrowed alike.
- The first discipline is telling **invented** claims (yours to command) from **borrowed** ones (which must be grounded).
- The **trust ladder** ranks ways of knowing from a model's guess up to a deterministic computation; the goal is to know — and record — which rung you stand on.
- Facts (trusted), Notes (speculative), and Sources (citations) are kept separate so you can rely on the Facts.
- **Provenance** — every kept fact remembering its origin — is the quiet habit that makes the whole corpus trustworthy.

CHAPTER 2

2

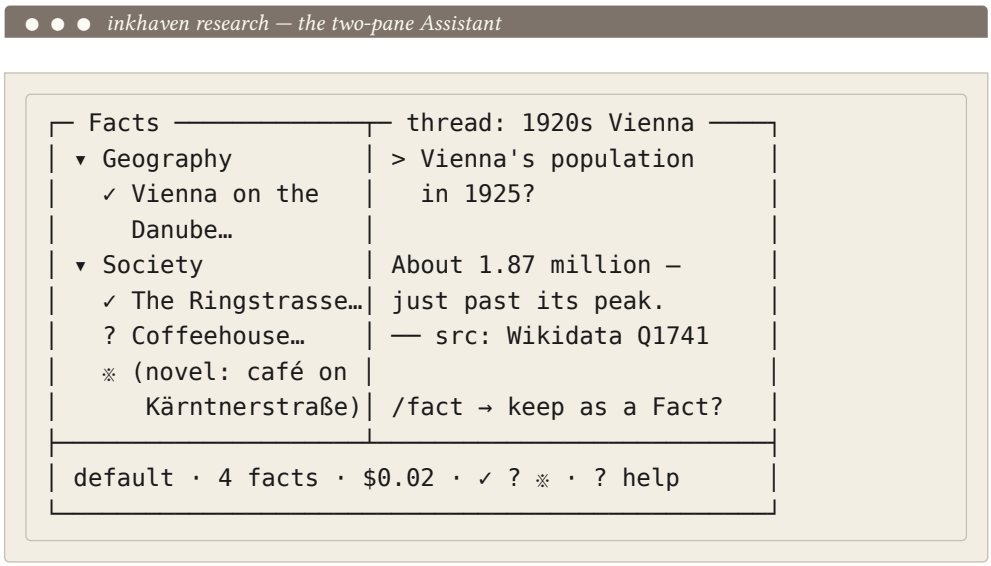
The Research Assistant

The Research Assistant is a separate screen inside Inkhaven — a room you step into when you want to research, and step out of to write. It is deliberately quiet: two panes, a place to type, and a status line. Everything you do here either **gathers** a fact, **checks** one, or **composes** from the ones you have kept.

This chapter is a tour of the room and the way you move through it. There is no command to memorise yet; the point is to understand the shape of the work so the commands in the next chapter feel like the natural thing to type.

Two panes, one conversation

Picture the screen split in two — the Facts tree on the left, the conversation on the right, a status line beneath:



On the left is your **Facts tree** — the growing outline of everything you have kept for this project: your Facts, organised the way a book is, in chapters and sections you can fold and unfold. When you are new to a project this pane is nearly empty. It fills as you work, and by the end of a book it is the map of everything you learned.

On the right is the **conversation** — where you ask questions and read answers. You type at the bottom; the exchange scrolls above. This is where the language model lives, where search results arrive, where a fact-check verdict appears. It is a chat, but a chat with a purpose: everything said here is a candidate for the tree on the left.

The whole workflow is the traffic between these two panes: a question on the right becomes, after checking and your confirmation, a fact on the left.

TERM **Thread**

A **thread** is one research conversation, saved. You might keep one thread per topic — "the aqueduct," "1920s Vienna," "cell biology for chapter 9" — so each line of inquiry has its own history you can return to. Threads keep unrelated research from tangling together, and they persist between sessions.

Two ways to speak

You say things to the Assistant in one of two registers, and learning the difference is most of learning the tool.

The first is **plain language**. You type a question the way you would ask a knowledgeable friend — "how far could a Roman legion march in a day?" — and you get an answer, grounded on whatever you have already gathered. This is for **thinking**: exploring, orienting, working out what you actually need to know.

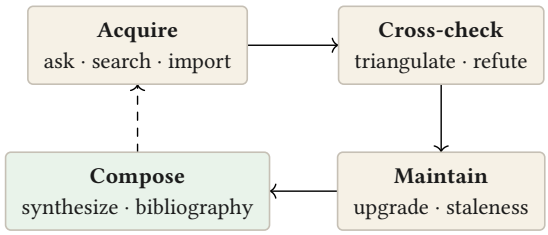
The second is a **command**. A command starts with a slash — `/fact`, `/geonames`, `/triangulate` — and it does something precise and repeatable: keep this claim as a fact, look this place up in a gazetteer, cross-check this against three sources. Commands are for **acting**: turning a loose exchange into something durable.

YOU DON'T HAVE TO MEMORISE THE COMMANDS

Type a single `/` and a **command palette** opens — a searchable list of every command with a one-line description. You can browse, filter by typing, and pick one without ever having learned its name. Throughout this book, when a command is introduced, that palette is where you would rediscover it later. Learn the workflow; let the palette remember the spelling.

The arc of a project

Zoom out from a single question to a whole book, and the work has a natural shape. It is a loop, and the Research Assistant is built to carry you around it:



The corpus you build is not a dead pile of notes — it is checked, kept fresh, and composed back out.

You **acquire** — ask, search, import — pulling raw material in. You **cross-check** — hold a claim up against independent sources, even argue against it — so that only what survives is kept. You **maintain** — because a knowledge base decays; guesses can be re-grounded on real sources later, and old facts can be flagged for a second look. And you **compose** — turning the corpus back into output: a synthesis, an outline, a bibliography that feeds the very book you are writing.

Most of this book is a slow walk around that loop. Part I gets you onto it — one question, one checked fact. Later parts deepen each quarter of the circle.

FICTION

A novelist tends to live in **acquire** and **compose**: gather enough real texture to make the world convincing, then let it quietly inform the prose. Cross-checking matters most for the load-bearing facts a sharp reader would catch.

NON-FICTION

A non-fiction author lives in **cross-check** and **maintain**: every claim must survive scrutiny and stay current, and the bibliography that **compose** produces is often part of the deliverable itself.

Nothing happens to your manuscript by accident

One promise, made early because it shapes how freely you can experiment: the Research Assistant never touches your manuscript on its own, and never edits your prose. It writes only to your research books — Facts, Notes, Sources — and only the deliberate acts you take (keeping a fact, recording a source) change anything on disk. You can ask anything, explore any tangent, take any command back out for a test drive, and know that your book on the other screen is exactly as you left it.

That safety is what lets the next chapter be hands-on. You are about to gather your first fact — and there is nothing you can do in the process that you cannot undo.

WHAT YOU LEARNED

- The Research Assistant is a separate, quiet screen: a **Facts tree** on the left, a **conversation** on the right, and the traffic between them is the workflow.
- **Threads** keep separate lines of inquiry apart and persist between sessions.
- You speak in **plain language** to think and explore, or in **slash commands** to act; a `/` palette means you never have to memorise a command.
- A project moves around a loop — **acquire → cross-check → maintain → compose** — and the tool is built to carry you around it.
- The Assistant writes only to your research books and never edits your prose, so you can explore without risk.

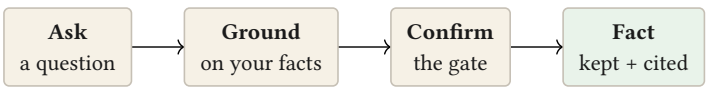
CHAPTER 3

3

Your First Fact

Enough groundwork. Let us put one real fact into your book, from question to kept claim, and watch what the Research Assistant does at each step. The whole thing takes about a minute; understanding it takes a chapter, because every stage is teaching you something you will lean on for the rest of the project.

Here is the shape of what we are about to do:



Every claim earns its place: nothing is written to your Facts without your say-so.

Ask a question. Ground the answer on what you already trust. Cross a confirmation step — the **gate** — where you get the final say. Keep the result as a fact that remembers where it came from. Four steps, and you will do them thousands of times.

Ask

Open the Research Assistant, and in the conversation pane type a plain question — whatever your book actually needs to know. Not a command, not a keyword; a real question.

FICTION

```
How far could a Roman legion
march in a single day on a
good road?
```

You need this because your protagonist has to reach the frontier "in time," and **in time** has to mean something.

NON-FICTION

```
What was the average daily
marching distance of a Roman
legion, and what is
   the evidence for it?
```

You ask for the **evidence** too, because your reader will expect a citation, not just a number.

The Assistant answers in prose. Behind the scenes it did something important before replying: it looked through the facts you have **already** kept for this project and offered them to the model as context, so the answer is consistent with your own established world — not a generic answer that might contradict what you decided three chapters ago.

TERM **Grounding on your corpus**

Before answering, the Assistant **retrieves** the most relevant facts you have already kept and hands them to the model along with your question. This is why its answers get better as your corpus grows: it is not guessing in a vacuum, it is reasoning over what **you** have established. Early on, with an empty corpus, it leans on the model's own knowledge — which is exactly why the next steps matter.

Turn the answer into a candidate fact

Suppose the answer says a legion covered roughly twenty Roman miles a day, more under forced march. That is useful — but right now it is just words in a chat, sitting on the bottom rung of the ladder: a model's guess. To keep it, you turn it into a candidate fact with a command:

```
/fact a Roman legion marched about 20 Roman miles (30 km) per day
```

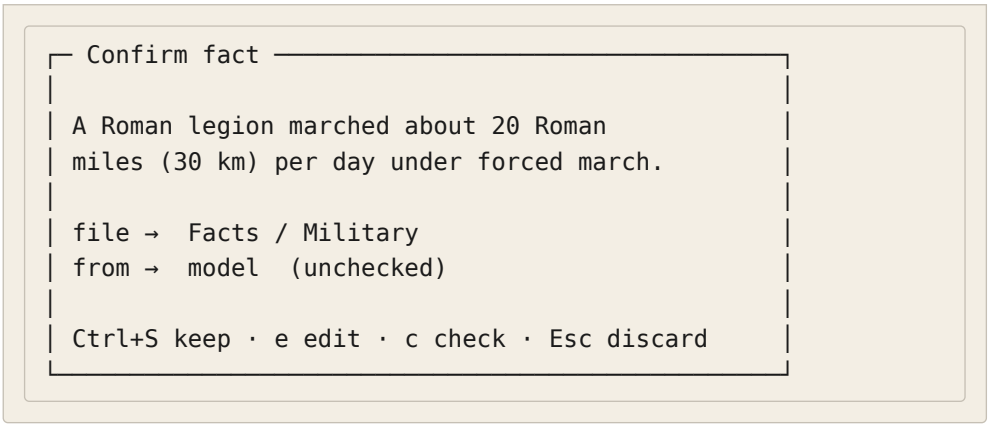
You do not have to retype the whole answer; you can point `/fact` at the claim you want and let it distil a clean, single statement. What you are doing is **promoting** a line from the disposable conversation into something the book will remember.

> **FACT OR NOTE?**
>
> If the claim is solid enough to build on, `/fact` puts it in your trusted Facts book. If it is a promising lead you are not sure about yet, its sibling `/note` puts it in Notes instead — the same gesture, a different shelf. You can promote a Note to a Fact later, once it has earned it. When in doubt, take a Note; the Facts book is worth keeping clean.

The gate: you get the last word

Here is the step that makes the whole tool trustworthy. `/fact` does not silently write to your book. It opens a **confirmation** — the gate — showing you exactly what is about to be kept: the fact's wording (which you can edit), where it will be filed, and where it says it came from.

The confirmation gate — nothing is kept until you say so



TERM **The confirmation gate**

The **gate** is the mandatory pause between "the Assistant proposed a fact" and "the fact is in your book." Nothing reaches your Facts without passing it. You can edit the wording, accept it with `Ctrl+S`, or discard it entirely. The Assistant proposes; you dispose.

For a plain model-derived claim like this one, the gate may also offer to **check** the fact before you commit — and for facts drawn from the web or from structured sources, later chapters show the gate doing real cross-checking here. For now the important thing is the principle: **a fact becomes yours only when you say so.** If the wording is off, fix it in the gate. If you have changed your mind, discard it and nothing is written. If it is right, confirm — and it lands on the tree to your left.

What you just kept

Look at the new fact in the Facts tree. It is not just a sentence. Attached to it, quietly, is its **provenance** — a small record that this fact came from the model (the bottom rung), together with the question that produced it. You can call that record up any time.

This matters more than it seems. That provenance is honest: it says, in effect, "this is currently only a guess." Later chapters will show you how to **climb the ladder** with this exact fact — re-grounding it on a real source with a single command, at

which point its provenance updates to say so, and the fact grows firmer without you rewriting a word of it.

FICTION

Your legion's march is now a kept fact your story can lean on — and one you have flagged (via its provenance) as still needing a real source before you fully trust the timeline that depends on it.

NON-FICTION

Your marching-distance claim is kept, but its provenance says **model** — which for your purposes is not good enough yet. You now know precisely which of your facts still need a citation, because each one tells you.

The habit under the habit

That is the entire core loop, and it never gets more complicated than this: ask, `/fact`, confirm. Everything else in this book makes the **middle** richer — better places to draw the answer from, real cross-checking at the gate, ways to keep the corpus healthy — but the gesture stays the same. Ask a question; keep what survives; let it remember where it came from.

In the next part we start climbing. Instead of grounding on the model's memory, you will draw facts from authoritative sources — structured knowledge bases, real places, scholarly papers, whole public-domain books — so that the facts you keep start their life much higher on the ladder.

WHAT YOU LEARNED

- The core loop is **ask → `/fact` → confirm**, and it never gets harder than that.
- A plain question is answered **grounded on the facts you already kept**, so answers improve as your corpus grows.
- `/fact` keeps a trusted claim; `/note` keeps a speculative one — when in doubt, take a Note and promote it later.
- The **gate** is a mandatory pause: you edit, confirm, or discard, and nothing reaches your Facts without your say-so.
- Every kept fact records its **provenance** — even "just a model's guess" — which is what lets you climb the ladder later without rewriting anything.

PART II

Authoritative Sources

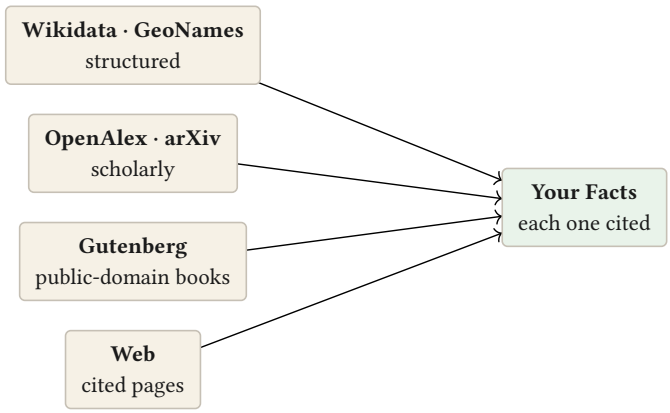
CHAPTER 4

4

Structured Facts and Real Places

In Part I you kept a fact that began life as a model's guess — the bottom rung. It worked, but its provenance admitted the truth: **this is only a guess.** This part is about starting higher. Instead of asking a model to recall a fact, you ask an authoritative source to **hand you one**, already carrying a citation.

There are several such sources, and the rest of this part introduces them one at a time. Here is the family:



Each source starts a fact higher on the ladder than a model's guess — and each one hands you a citation for free.

We begin at the top of that fan — the **structured** sources — because they are the firmest of the ones you look up, and because they show most clearly what "authoritative" buys you.

Prose versus a datum

Most of what you find when you research is **prose**: a paragraph that mentions the fact you want, surrounded by other words, written by someone with a point of view. You have to read it, extract the fact, and trust the writer.

A structured source gives you something different: a **datum**. Not "an article that discusses the founding of Rome," but the discrete, identified statement **Rome — inception — 753 BCE**, attached to a stable identifier that a reader can resolve independently.

> TERM **Structured data**
>
> **Structured data** is information stored as discrete, labelled facts rather than free-flowing prose — subject, property, value — each with a stable identifier. Because a structured fact is already broken into its parts and carries an id, there is nothing to misread and everything to cite.

This is why the structured rung sits so high on the ladder. There is no paragraph to misinterpret, no author's slant to see past. The fact arrives pre-checked and pre-cited.

Wikidata: facts with an id

The first structured source is **Wikidata** — a vast, curated database of facts about people, places, works, events, and things, each identified by a code. You reach it with a command:

```
/wikidata Roman aqueduct
```

The Assistant looks the entity up and shows you a compact card: the thing's name, a short description, and its key properties as clean subject–property–value lines — cited by the entity's id.

> **TERM Q-id**
>
> Every Wikidata entity has a **Q-id** — a stable code like `Q220` (the city of Rome). It never changes, so a fact cited by Q-id can be re-checked years later. When you keep a `/wikidata` fact, that Q-id becomes its provenance.

Take a fact from that card — `/fact`, exactly as before — and something new happens at the gate: because the source is structured and authoritative, the fact **skips the check**. There is nothing to fact-check; a Q-id-backed datum is already a verifiable fact, not a guess to be second-guessed. It lands in your Facts with its provenance reading `wikidata`, several rungs above where the same fact would have started from the model.

> **WHY NOT JUST USE AN ENCYCLOPEDIA?**
>
> Inkhaven deliberately grounds on Wikidata's **per-statement data**, not on encyclopedia prose. Narrative articles carry well-documented editorial slant — in politics, economics, history — that discrete triples do not. The tool grounds on the fact, not the story told around it. It is a small design choice that keeps the low-slant sources doing the load-bearing work.

GeoNames: the world's places

The second structured source is a **gazetteer** — a dictionary of real places. Fiction and non-fiction alike lean constantly on geography: where a town is, what region it sits in, how large it was, what kind of place it is. `/geonames` answers all of that from the GeoNames database:

```
/geonames Rome
```

You get a card — the place's name, its region and country, its type (**capital of a political entity**, **populated place**, **stream**), its coordinates, its population — cited by a GeoNames id. Like Wikidata, it is structured, so a `/fact` from it skips the check and records `geonames` provenance.

> TERM **Gazetteer**
>
> A **gazetteer** is a geographical dictionary: a database of place names with their locations, administrative regions, feature types, and populations. It answers "where is this, and what kind of place is it?" with real coordinates rather than a guess.

Coordinates are quietly powerful. Once a place is a fact with real latitude and longitude, later chapters can **compute** with it — the distance between two towns, whether a day's travel between them is plausible — turning a looked-up place into a rung-topping computed fact.

> FICTION
>
> Ground your invented map in a real one. Give your fictional city the coordinates and climate-band of a real analogue, and its distances and seasons will quietly behave — the reader feels a world that holds together, without ever seeing a source.

> NON-FICTION
>
> Every place-name in your text becomes citable — region, type, population, all pinned to a GeoNames id a reader can resolve. Your geography stops being assertion and becomes evidence.

ONE-TIME SETUP FOR GEONAMES

GeoNames asks for a **free username** (a one-time registration, not a paid key). Until you set `research.geonames.username`, `/geonames` politely reports that it is unavailable — nothing breaks, the command simply waits for its username. Wikidata needs no setup at all.

Starting higher

Notice what changed between Part I and this chapter. The **gesture** is identical — ask, `/fact`, confirm — but the fact now begins its life several rungs up, cited, and gate-skipped, because the source vouched for it. You did less second-guessing, and you got a citation you never had to write.

The structured sources cover an enormous amount of what a book needs: dates, identities, relationships, places, populations. But not everything is a datum in a database. For claims that live in the **literature** — findings, studies, arguments — you want the scholarly rung. That is the next chapter.

WHAT YOU LEARNED

- Authoritative sources **hand you a cited fact** instead of a paragraph to interpret — starting it high on the ladder.
- **Structured data** is discrete labelled facts with stable ids; there is nothing to misread and everything to cite.
- `/wikidata` returns facts by **Q-id**; a `/fact` from it skips the check and records `wikidata` provenance.
- `/geonames` returns real places from a **gazetteer** (region, type, population, coordinates), cited by id; coordinates enable later computation.
- The workflow gesture is unchanged — **ask, `/fact`, confirm** — but the fact starts cited and gate-skipped.

CHAPTER 5

5

The Literature and the Library

Some facts are not data points — they are **findings**. The result of a study, the argument of a paper, the passage in a book. For these the structured sources of the last chapter have nothing to hand you; you need the **literature** and the **library**. This chapter adds three sources: two scholarly indexes and a shelf of seventy-five thousand public-domain books.

Scholarly work: `/openalex` and `/arxiv`

The scholarly record is where careful claims live, and Inkhaven reaches two large, free indexes of it.

`/openalex` searches OpenAlex — a comprehensive index of scholarly works across every field — and returns the top matching paper: its title, authors, year, venue, and abstract, identified by a DOI.

`/arxiv` does the same against arXiv, the preprint server for the sciences — useful when you want the newest work, before it has cleared peer review.

```
/openalex Roman aqueduct hydraulic capacity
```

> TERM **DOI**
>
> A **DOI** (Digital Object Identifier) is a permanent code that points to a specific scholarly work — the scholarly equivalent of a Q-id. A claim cited by DOI can be resolved to the exact paper by any reader, forever.

> TERM **Preprint**
>
> A **preprint** is a scholarly paper shared publicly **before** formal peer review. It is fast and current, but less settled than a published article — worth citing with that caveat in mind. arXiv is the largest preprint server.

A `/fact` taken from a scholarly result records `openalex` or `arxiv` provenance, with the paper's identifier — and it does one more thing for you automatically.

The citation files itself

When you keep a fact grounded on a paper, the Assistant also writes a proper bibliography entry — author, title, year, DOI — into your **Sources** book, under a Research chapter, without you lifting a finger. You research; the bibliography assembles itself in the background. A much later chapter will show you turning that accumulated Sources book into a formatted reference list with a single command. The habit that makes it possible starts here: every scholarly fact you keep quietly deposits its citation.

GROUNDING ON THE PAPER, HONESTLY

A scholarly `/fact` grounds on the paper's **metadata and abstract** — enough to cite the claim and point a reader to the work. It is not a promise that you read the full text. For claims where that matters, the next section — ingesting a whole source — lets you draw facts from the actual pages.

The library: `/gutenberg`

Project Gutenberg is a library of some seventy-five thousand **public-domain** books — out of copyright, free to read and reuse. Inkhaven can pull one straight into your corpus:

```
/gutenberg pride and prejudice
```

It searches the catalogue, fetches the book's text, strips the boilerplate, and **ingests** it — breaks it into passages and files them as research material, credited to the book. From then on, when you ask a question, the relevant passages of that book can be retrieved and quoted back to you, cited as `[source: <title>]`.

TERM **Ingesting a source**

To **ingest** a source is to bring its full text into your corpus, split into searchable passages. Afterwards, the Research Assistant can **retrieve** the passages relevant to a question and ground an answer on them — so a fact can quote the actual text, not a summary of it. This is how a whole book becomes something you can search and cite.

Because Gutendex (the catalogue behind `/gutenberg`) searches by **title, author, and subject**, you find the book by its metadata; the passage-level searching happens afterwards, inside your own corpus. If the top hit is not the edition you want, the reply lists alternatives by their catalogue number, and you can ingest a specific one — or even a single chapter of a long book — by asking for it directly.

FICTION

Ingest a period text — a Victorian novel, a real memoir, a public-domain history — and let its **voice and texture** inform your prose. Ask "how did they describe a railway journey in 1870?" and get actual period passages to steep in, quoted and credited.

NON-FICTION

Ingest a primary source — a public-domain treatise, a historical document — and quote from it **by page**, cited. A claim grounded on an ingested source points the reader at the exact text, not a paraphrase.

PUBLIC DOMAIN, USED POLITELY

Gutenberg texts carry no copyright; ingesting them is unproblematic. The catalogue is a free service, and Inkhaven fetches one book per command with a polite cap on how much of a very long book it embeds — enough to be useful, bounded enough to be a good citizen. It all degrades cleanly when you are offline.

Where each source sits

You now have the whole authoritative fan except the web (next chapter). It helps to hold them in one view — same gesture, different rung:

- `/wikidata`, `/geonames` — structured data, cited by id, gate-skipped (Chapter 4).
- `/openalex`, `/arxiv` — the scholarly record, cited by DOI, auto-filed to your bibliography.
- `/gutenberg` — full public-domain books, ingested so their passages are searchable and quotable.

Each one starts a fact higher than a guess and hands you a citation. What they share is authority you can point at. The web — vast, immediate, uneven — is the one source where that authority has to be **earned** at the gate. That is the last piece of the fan, and the next chapter.

WHAT YOU LEARNED

- `/openalex` and `/arxiv` return scholarly papers cited by **DOI**; arXiv adds current **preprints**, with the peer-review caveat.
- A scholarly `/fact` **files its own citation** into the Sources book — the bibliography assembles itself as you research.
- `/gutenberg` **ingests** whole public-domain books; their relevant passages are then retrieved and quoted, cited by title.
- **Ingesting** means the full text becomes searchable in your corpus, so a fact can quote the actual pages rather than a summary.
- All the authoritative sources share one thing: they hand you a citation and start the fact high on the ladder.

CHAPTER 6

6

The Web, Earned

The structured sources and the scholarly indexes are authoritative by construction — a Q-id, a DOI, a catalogued book. The open web is not. It is vast and immediate and often exactly what you need, but its authority is uneven: a careful reference page and a careless blog post look the same until you read them. So Inkhaven treats a web fact differently from every other source. It lets you use the web freely — and then makes a web fact **earn** its place at the gate.

Asking the web

The command is `/web`:

```
/web how wide was a standard Roman road
```

The Assistant searches, fetches the actual pages, and grounds an answer on what they say — cited by URL. This is a real step up from the model's unaided memory: there is now a **page** behind the claim, one you (or a reader) can open. On the ladder, a cited web page sits above a guess but below the structured and scholarly rungs — because a URL proves **where** a claim was found, not that it is **true**.

The fact-check gate

Here is what makes the web safe to lean on. When you take a `/fact` from a web-grounded answer, the confirmation gate does something it did not do for a structured source: it **checks the claim before letting you keep it.**

> **TERM The fact-check gate**
>
> For a fact drawn from the web (or from the model), the confirmation gate runs a single-claim accuracy check **before** the fact commits. An `ACCURATE` verdict lets it through; a `DUBIOUS` or `INACCURATE` verdict shows you the reasoning and asks you to confirm again — deliberately, with your eyes open. It informs; it never silently blocks. You always get the final say.

This is the same gate from Chapter 3, doing more work. For a Wikidata datum there was nothing to check. For a web page — which might be wrong — the gate pauses, weighs the claim, and reports back. If it is confident the claim is accurate, the fact lands normally. If it has doubts, it does not throw the fact away; it **tells you why** and asks whether you want to keep it anyway. A shaky claim can still be kept — sometimes you know better than the check — but you keep it knowingly, and its provenance records the verdict.

> **INFORM, NEVER BLOCK**
>
> Every gate in Inkhaven follows the same principle: it surfaces what it found and hands you the decision. It will warn, it will double-check, it will ask you to confirm twice — but it will never refuse to record a fact you insist on. You are the author; the tool is your research assistant, not your gatekeeper.

Two ways to use a page

`/web` has two modes, and knowing which you want is the whole skill.

The default is what we just described: **ground a cited answer** on the fetched pages, and let you `/fact` from it through the checking gate. Use this when you want an **answer** — a specific claim, checked and kept.

The other mode, `/web --ingest`, **embeds the pages themselves** into your corpus — the same ingestion you met with `/gutenberg`, but from the live web. No model answers; the pages simply become searchable, quotable research material. Use this when you want the **material**, not a single answer — when you are gathering a body of reading to draw on repeatedly.

FICTION

Chase texture and detail quickly — the smell of a trade, the layout of a period street — grounding on real pages, keeping only the details that survive the check. The reader never sees the URLs; they feel the specificity.

NON-FICTION

Pull a claim from a reputable page **with the check as a first filter**, then — in the next part — cross-check the survivors against the structured and scholarly rungs before the claim goes into your argument. The web is where a non-fiction claim **starts**, not where it rests.

A note on the low rungs

It is worth saying plainly: there is nothing wrong with using the web, or even the model. The ladder is not a rule that forbids the lower rungs — it is a way of being **honest** about where a fact stands. A novelist grounding the feel of a city on a good web page has done real, legitimate work. The check at the gate, and the provenance on the fact, simply keep that work truthful about itself.

What the ladder **does** insist on is this: for a load-bearing claim — the kind a sharp reader or a reviewer would test — a single web page is a starting point, not an ending. The next part is about turning a starting point into an ending: holding

a claim up against several independent sources at once, and even arguing against it, so that what you finally keep has been tested from more than one side.

WHAT YOU LEARNED

- `/web` searches and grounds a **cited** answer on real pages — above a guess, but below the structured and scholarly rungs, because a URL proves **where**, not **whether**.
- Taking a `/fact` from a web answer runs the **fact-check gate**: `ACCURATE` passes; `DUBIOUS` / `INACCURATE` shows its reasoning and asks you to confirm again.
- Every gate **informs but never blocks** — you can always keep a fact you insist on, knowingly, with the verdict recorded in its provenance.
- `/web --ingest` embeds whole pages into your corpus as searchable material, instead of answering a single question.
- The low rungs are legitimate; the ladder just keeps a fact honest about where it stands — and load-bearing claims deserve the cross-checking of Part III.

PART III

Trust & Cross-Checking

CHAPTER 7

7

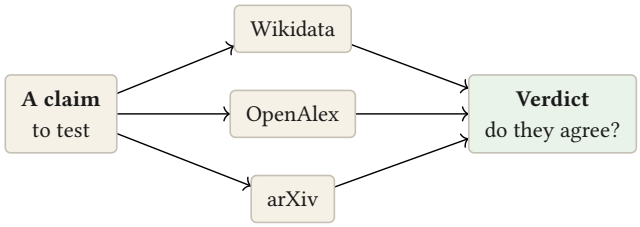
Cross-Checking a Claim

A single source can be wrong. A reputable web page can repeat a myth; a paper can be superseded; even a structured database can carry a stale value. The most reliable thing you can do with a load-bearing claim is not to find a **better** single source — it is to ask **several independent sources at once** and see whether they agree. That is triangulation, and it is the heart of this part.

The idea: agreement, not authority

When three surveyors fix a point from three directions, no single line locates it — the **intersection** does. A claim works the same way. A fact that Wikidata, a

scholarly paper, and a preprint all support — with none of them contradicting it — is far firmer than any one of them alone, because for it to be wrong, several independent records would all have to be wrong in the same way.



*A claim the sources **agree** on, with none contradicting, is far firmer than any single one alone.*

> TERM **Triangulation**
>
> **Triangulation** is testing a claim by gathering evidence from several **independent** sources at once and judging whether they agree. The verdict is not "a source says so" but "the sources **concur**" — which is a much stronger thing to be able to say.

The `/triangulate` command

Point `/triangulate` at a claim — or run it bare to test the last answer in the conversation:

```
/triangulate the aqueduct carried about a million cubic metres of
water per day
```

The Assistant queries the structured and scholarly sources **concurrently**, gathers what each one says, and then judges the evidence — reporting, per source, whether it **supports**, **contradicts**, or is **silent** on the claim, and a bottom line of how many concur.

```
Wikidata: SILENT — no capacity statement for this entity
OpenAlex: SUPPORTS — the cited work gives a comparable daily
figure
arXiv:    SILENT — the preprint models flow, not capacity
Agreement: 1/3 support, 0 contradict
```

The important discipline: the model here is judging **external evidence**, not grading its own earlier answer. That is what makes the verdict mean something. "The sources I can reach don't corroborate this" is a genuinely useful thing to learn **before** you commit a claim your argument will rest on.

> **SILENCE IS INFORMATION**
>
> A `SILENT` source has not failed — it simply has nothing to say about this particular claim. Triangulation only counts **support** and **contradiction**; a claim that no source can speak to is one you have learned you cannot yet ground by cross-checking, which is worth knowing on its own.

Triangulation as the gate

You can run `/triangulate` by hand whenever a claim feels shaky. But you can also make it **automatic**. Turn on the setting `research.triangulate_gate`, and from then on every `/fact` drawn from a low rung — a model guess, a web page, an imported document — is triangulated **before it commits**. Cross-source agreement becomes the gate.

At the confirmation, a claim that the sources **support with no contradiction** inserts normally. A claim that is **contradicted** or that **no source corroborates** shows you the agreement summary and asks you to confirm again — the same inform-never-block principle you met with the web fact-check, now backed by multiple independent references instead of one. And because the structured and scholarly facts (`wikidata`, `geonames`, `openalex`, `arxiv`) are already authoritative, they skip the gate entirely — there is nothing to cross-check about a datum that arrived cited.

FICTION

Reserve triangulation for the **load-bearing** facts — the date the reader could catch, the distance the plot depends on. You don't need three sources for the colour of a market awning; you very much want them for the thing a sharp reader will test.

NON-FICTION

Make triangulation the default. Turn the gate on and let every borrowed claim face cross-source agreement before it enters your argument. A claim that survives triangulation is one you can footnote with confidence.

What triangulation cannot do

Triangulation asks **who agrees**. It is a corroboration test — it looks for support. That is powerful, but it has a blind spot: a plausible-sounding false claim that the sources happen to be **silent** on will pass through uncorroborated rather than caught, and a claim can be **agreed upon and still wrong** if the sources share a common error.

So the next chapter turns from checking one claim to auditing the **whole** Facts book — looking for internal contradictions across everything you have kept — and the chapter after that adds the mirror image of triangulation: a check that does not ask who agrees, but actively tries to **disprove** the claim. Corroboration and refutation are two different questions, and a fact you can lean on has answered both.

WHAT YOU LEARNED

- **Triangulation** tests a claim against several **independent** sources at once and judges **agreement**, not mere authority.
- `/triangulate` queries the structured + scholarly sources concurrently and reports each as SUPPORTS / CONTRADICTS / SILENT, with a concurrence tally.
- The model judges **external evidence**, not its own answer — which is what makes the verdict trustworthy; `SILENT` is information, not failure.
- `research.triangulate_gate` makes cross-source agreement the automatic gate for low-rung `/fact` s; structured/scholarly facts skip it.
- Triangulation asks **who agrees** — it can't catch an agreed-upon error or a plausible claim the sources are silent on; refutation (Chapter 9) is its mirror.

CHAPTER 8

8

Auditing Everything You Kept

So far you have checked claims one at a time, as you kept them. But a knowledge base has a property no single fact has: **internal consistency**. Two facts, each plausible on its own, can quietly contradict each other — a date in Chapter 3 that cannot be reconciled with a journey in Chapter 9, a population that does not match a later figure. As a corpus grows past what you can hold in your head, you need a way to check not just each fact, but **all of them together**. That is `/factcheck`.

Two questions at once

`/factcheck` audits your Facts book along two axes.

The first is **truth**: taking each fact on its own and asking whether it holds up — the same kind of accuracy check the gate runs, now swept across everything you have kept, so a fact that slipped through months ago gets a fresh look.

The second is **consistency**: taking your facts in related groups and asking whether they **agree with each other** — surfacing the internal contradictions that no single-fact check could ever see, because the problem is not in either fact but in the **pair**.

> TERM **Audit**
>
> An **audit** is a pass over your whole Facts book at once, rather than a check of a single claim as you write it. It catches two things a one-at-a-time workflow misses: old facts that were never re-examined, and **contradictions between** facts that are each fine alone.

You run it when a draft is far enough along that consistency matters — before you hand a manuscript to a reader, an editor, or a reviewer. It is the research equivalent of a spell-check pass: not something you do on every sentence, but something you would not ship without.

Reading the verdicts

An audit is only useful if you can **see** its results without reading a report. After `/factcheck` runs, each fact in your tree wears a small verdict mark, so the state of your whole corpus is legible at a glance:

- ✓ — the fact held up. Nothing to do.
- **?** — the fact is **dubious**; the audit had doubts worth your attention.
- ✕ — the fact did not pass; something is wrong or contradicted.

> TERM **Verdict**
>
> A **verdict** is the audit's judgement on one fact, shown as a glyph beside it in the tree — passed (✓), dubious (?), or failed (✕). The glyph is a standing reminder: it persists after the audit, so you can work through the doubtful and failed facts at your own pace, and see at a glance which parts of your corpus are solid.

The glyphs turn a wall of text into a map. Green-check branches are settled; question-marks and crosses are your worklist. You are never handed a verdict you cannot act on, and you are never forced to act on one immediately — the marks wait for you.

`/whatswrong`: why a fact failed

A ✕ tells you **that** a fact failed; it does not, by itself, tell you **why**. For that, select the flagged fact and ask:

```
/whatswrong
```

The Assistant explains — specifically and concretely — what it believes is inaccurate or contradicted about that fact, and what the correct information appears to be. Now the cross is actionable: you can fix the wording, re-ground the fact on a better source, or — if you know the check is mistaken — leave it, having looked. As always, the tool explains and recommends; the decision to change a fact is yours, and it never edits your prose to make it.

FICTION

Run an audit before a beta read. The ✕ and ? marks catch the continuity slips that pull a reader out of the story — the mismatched date, the town that moved sixty miles between chapters — while they are still cheap to fix.

NON-FICTION

Run an audit before submission or review. The consistency pass is exactly what a hostile reader will do to your argument; better that you find the contradicted figure than a referee does. `/whatswrong` turns each failure into a specific correction.

THE AUDIT INFORMS; YOU DECIDE

Like every check in this book, `/factcheck` never changes a fact on its own. It marks, it explains, it recommends. A ✕ you disagree with can stay — sometimes the audit is wrong, or the "contradiction" is a deliberate feature of your world. The glyph simply guarantees you **saw** it.

From finding faults to fixing tiers

An audit tells you which facts are shaky. Often the fix is not to delete a fact but to **strengthen** it — to take a claim that is only a model's guess and re-ground it on a real source, moving it up the ladder. The Assistant can do that for you, almost automatically, and it can also warn you when a fact has simply grown old. Strengthening and maintaining what you have kept is the subject of the next chapter — along with the mirror-image of triangulation promised earlier: a check that tries its hardest to **disprove** a claim before you trust it.

WHAT YOU LEARNED

- `/factcheck` **audits the whole Facts book** along two axes — per-fact **truth** and cross-fact **consistency** — catching contradictions no single-fact check can see.
- Run it at draft milestones (before a beta read, an edit, a submission), like a spell-check pass for your facts.
- Each fact then wears a **verdict** glyph — ✓ passed, ? dubious, ✕ failed — turning your tree into a legible worklist that waits for you.
- `/whatswrong` explains **why** a flagged fact failed and what the correct information is, making each ✕ actionable.
- The audit marks and explains but never edits — and often the right fix is to **strengthen** a shaky fact, which is the next chapter.

CHAPTER 9

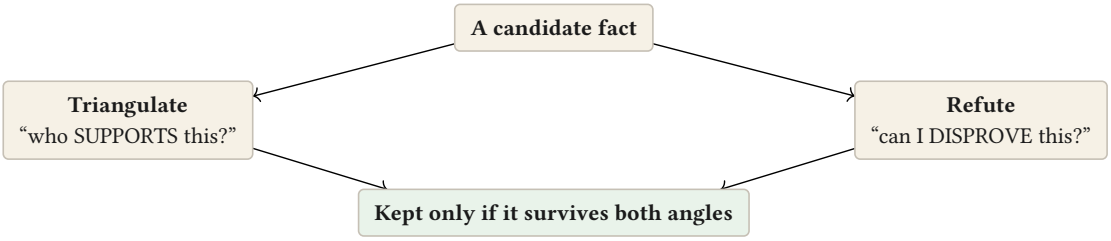
9

Strengthening What You Keep

Two chapters ago, triangulation left us with a promise: it asks **who agrees**, and a claim can slip through simply because the sources are silent on it. This chapter delivers the mirror image — a check that does not look for support but actively tries to **knock the claim down** — and then turns from **vetting** new facts to **maintaining** old ones: strengthening a guess into a cited fact, and flagging facts that may have gone stale. This is the "maintain" quarter of the project arc.

The mirror of triangulation: refutation

Corroboration and refutation are two different questions. Triangulation asks "which sources support this?" Refutation asks "can I **disprove** this?" — and the difference matters, because a plausible, fluent, wrong claim is exactly the kind that attracts no contradiction and simply sits there looking reasonable. To catch it, you have to **attack** it.



Corroboration asks who agrees; refutation attacks the claim. A fact that passes both is one you can lean on.

> TERM **Refutation**
>
> **Refutation** is a check that actively tries to **disprove** a claim — hunting for factual errors, anachronisms, internal contradictions, things that conflict with well-established knowledge. Where triangulation looks for agreement, refutation looks for the flaw. A claim that survives a genuine attempt to break it has earned more trust than one that merely went unchallenged.

Turn on `research.refute_gate`, and a plain model- or document-derived `/fact` — one not already handled by triangulation or the web check — gets a single sceptic's pass before it commits. The Assistant tries to refute the claim and reports a verdict: `SOUND` (it could not disprove it) lets the fact through; `REFUTED` shows you the reasoning and asks you to confirm again. It is cheap, it needs no external source, it works offline — and it is the natural check for the speculative tier, where you most need a second, adversarial opinion. Like every gate, it is advisory: you always keep the last word.

TWO QUESTIONS, ONE HABIT

You do not have to choose between triangulation and refutation — they answer different questions and compose. A load-bearing claim ideally survives both: the independent sources corroborate it, **and** a determined sceptic could not knock it down. When a fact has passed both, you are as sure as this workflow can make you.

Climbing the ladder: `/upgrade`

Back in Part I you kept a fact that was only a model's guess, and its provenance said so. The honest thing to do with such a fact, eventually, is to **ground it on something firmer** — and you should not have to delete it and start over to do that. `/upgrade` does it in place.

Select a `model`-origin fact and run:

```
/upgrade
```

The Assistant takes the claim to the structured and scholarly sources — the same triangulation engine from Chapter 7 — and asks whether any of them **corroborate** it. If one does, and none contradicts, it **raises the fact's provenance to that source**: the fact that used to say "just a guess" now says "corroborated by Wikidata," and its rung on the ladder climbs accordingly.

TERM **Tier upgrade**

A **tier upgrade** re-grounds an existing fact on a firmer source and raises its provenance to match — **without changing the fact's wording**. A guess becomes a cited fact over time. Only the provenance moves; your text is never touched.

This is the quiet engine of a corpus that improves with age. Early in a project you keep quick guesses to keep moving; later, in a maintenance pass, you `/upgrade` the ones your book leans on, and watch the low rungs of your Facts book climb — each fact growing firmer without a word of it changing.

FICTION

Draft fast on the model's memory to keep the story moving, then, before you hand the book over, `/upgrade` the handful of facts the plot actually depends on. The guesses that survive become cited; the ones that don't, you rewrite.

NON-FICTION

`/upgrade` is how a working bibliography fills in. Each speculative claim you re-ground pulls a real citation up under it — turning a placeholder into a footnote you can defend, one fact at a time.

Facts grow old: `/stale`

The last piece of maintenance is time itself. A fact grounded on a web page two years ago may no longer be current; a figure that was right when you kept it may have moved. `/stale` finds those:

```
/stale 90
```

It lists your `model`- and `web`-tier facts older than the number of days you give — the ones whose grounding is softest and most likely to have drifted — so you can re-verify or `/upgrade` them. Structured and computed facts, being firmer and more stable, are left out; there is little point re-checking a Q-id.

TERM **Staleness**

A fact is **stale** when enough time has passed that its grounding may no longer hold — a moving figure, a page that changed, a claim overtaken by newer work. `/stale` surfaces the soft, aging facts for a second look; it is how a knowledge base stays honest across the months a book takes to write.

The corpus that tends itself

Put the last three chapters together and you have the whole "trust" half of the work. You **triangulate** a claim against independent sources; you **refute** it to see if it survives attack; you **audit** the whole corpus for internal contradictions; you

upgrade guesses into cited facts; and you flag the **stale** ones for renewal. None of it edits your prose, all of it informs rather than dictates, and every step leaves the provenance more honest than it found it.

A corpus maintained this way is not a dead pile of notes — it is checked, current, and firm where it needs to be. Which means it is finally ready to be **used**. The next parts turn from building and tending the knowledge base to **computing** new facts from it, protecting the facts you **invented**, and composing the whole thing back out into the book you are writing.

WHAT YOU LEARNED

- **Refutation** is triangulation's mirror: it tries to **disprove** a claim rather than find support, catching plausible-but-wrong facts that attract no contradiction.
- `research.refute_gate` runs a cheap, offline sceptic pass on low-rung `/fact`s: `SOUND` passes, `REFUTED` asks again — advisory, never blocking.
- `/upgrade` re-grounds a `model` fact on a corroborating source and **raises its provenance tier in place** — the wording never changes, only the rung.
- `/stale [days]` surfaces aging `model`/`web` facts whose grounding may have drifted, for re-verification or upgrade.
- Together — triangulate, refute, audit, upgrade, refresh — they make a corpus that tends itself, honestly, without ever touching your prose.

PART IV

Computed Facts

CHAPTER 10

10

Facts You Compute

Not every fact is looked up. Some you **work out**. The distance between two towns is not something to find in a database — it follows from their coordinates. A day's ride is not a citation — it is a speed times a time. The compound growth of a population, the length of a year on an invented planet, the water a channel of a given size can carry: these are **derived** facts, and they sit at the very top of the trust ladder for a simple reason.

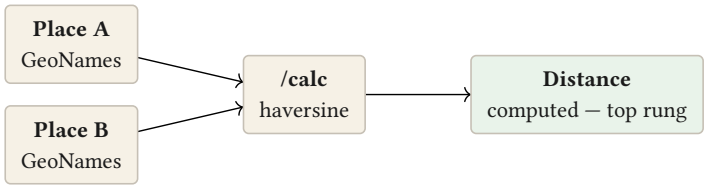
> **TERM Deterministic**
>
> A fact is **deterministic** when it follows from its inputs by a fixed rule, so that anyone who runs the same computation gets the same answer. A looked-up fact asks you to trust a source; a deterministic fact asks you to trust **arithmetic** — which is why it is the firmest rung of all. Nothing to misread, nothing to go stale, nothing to cite but the calculation itself.

The calculator that speaks your domains: `/calc`

`/calc` evaluates an expression and can turn the result straight into a fact. It is not just a four-function calculator — it knows the domains a book actually needs: unit conversions, geography, astronomy, climate, economics.

```
/calc 100 mi to km
```

Convert Roman miles to kilometres so your distances are consistent. Compute the great-circle distance between two sets of coordinates — the **haversine** — so a journey's length is real rather than guessed. Grow a figure by a rate over years with compound interest. Reduce a list of numbers to a sum or an average. In each case the result comes back as a value you can keep with `/fact`, and its provenance reads `computed` — the top rung — because the answer is not a claim anyone vouched for but a calculation anyone can repeat.



Two looked-up places become one derived fact — the firmest kind, because anyone can re-run the sum.

Notice how the sources compose. In Chapter 4 you learned two places from GeoNames, each with real coordinates. Here you feed those coordinates to `/calc` and get the distance between them — a **new** fact, firmer than either input, that you never had to look up. Looked-up facts become the inputs to computed ones, and the computed ones sit higher on the ladder than the sources they came from.

LET THE PALETTE HOLD THE SPELLINGS

`/calc` understands a broad vocabulary of operations — conversions, distances, growth curves, list reductions, domain formulas for climate and geography and economy. You do not need to memorise them. As with every command in this book, the way you rediscover an operation is to reach for it when you need it; the point here is the **idea** — that a fact can be derived, and that a derived fact is the firmest kind.

FICTION

Keep your invented world internally consistent by **computing** its constants, not guessing them. Fix your planet's year, its seasons, the distances on your map — once, deterministically — and every later scene that depends on them stays true, because they all trace back to the same arithmetic.

NON-FICTION

Do the quantitative work of your argument **in** your corpus, cited to the computation. A growth rate, an aggregate, a distance — each becomes a `computed` fact a reader can re-derive, which is the strongest form a quantitative claim can take.

A world that produces its own facts

For writers building an invented world — a planet, a continent, an imagined society — Inkhaven can go one step further than a calculator. It can **simulate** the world and materialise the results as facts you can research like any other.

TERM **The World book**

The **World** book holds a project's own deterministic simulation, compiled from a small definition: a star, a planet, its moons. From that, Inkhaven derives an astronomy (the length of the year in planet-days, axial tilt, seasons, lunar cycles, tides), a climate, a geography, a rough demography — and files them as facts. Because they are computed from a seed, they are consistent with each other and reproducible: the same world definition always yields the same world.

Once a world is compiled, `/world` lets you browse those simulated facts the way you browse any part of your corpus, and `/calc` can **read** them — computing new answers from your world's own constants. A `/fact` taken from the World book records `simulation` provenance: another top-of-the-ladder rung, deterministic for exactly the same reason `computed` is — re-run the simulation and you get the same world.

> **INVENTED, BUT NOT ARBITRARY**
>
> A simulated world fact is **invented** in the sense that the world is yours — but it is not **arbitrary**. Because it is computed from a seed by fixed rules, your invented planet's seasons and distances hold together as rigorously as real ones. This is the bridge between the two halves of your book: the facts you make up and the facts you borrow, both made trustworthy by the same discipline.

The top of the ladder

You have now met every rung. At the bottom, a model's guess; above it the web, your documents, the scholarly record, the structured databases; and at the top, the facts you **compute** and the facts your **simulation** produces — deterministic, reproducible, citing nothing but the calculation. A book grounded from top to bottom of this ladder, with every fact recording its rung, is about as honest as a book can be about what it knows.

Which raises the question the next part answers. Everything so far has been about **borrowed** facts — the ones a reader could check. But a work of fiction is built on facts the author **invented**, which must not be fact-checked at all. How does a tool this insistent on grounding make room for the things you simply decreed to be true? That is the subject of Part V.

WHAT YOU LEARNED

- Some facts are **derived**, not looked up — distances, travel times, growth, aggregates — and a **deterministic** fact is the firmest rung because anyone can re-run it.
- `/calc` computes across real domains (conversions, geography, astronomy, climate, economy); a `/fact` from it records `computed` provenance.
- Looked-up facts **compose**: GeoNames coordinates feed `/calc` to produce a distance firmer than either input.
- The **World** book compiles a project's own deterministic simulation into facts; `/world` browses them, `/calc` reads them, and a `/fact` from it records `simulation` provenance.
- Computed and simulated facts sit at the **top** of the ladder — invented worlds made rigorous by the same discipline as borrowed facts.

PART V

Fiction's Own Facts

CHAPTER 11

11

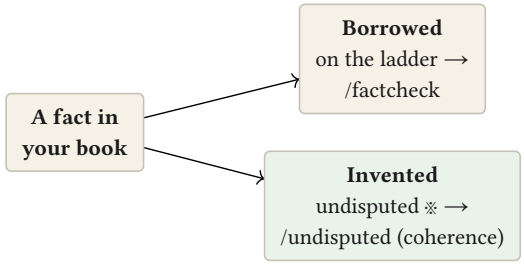
The Facts You Invented

This whole book has been about grounding — about never letting a claim rest on nothing. But a work of fiction is built on claims that rest on nothing **by design**. That the kingdom has three moons, that the drug wears off in an hour, that your detective was born in Lyon — these are not facts you got wrong or right. You **decreed** them. There is no external world to check them against, and a tool that tried to fact-check them would be making a category error.

So Inkhaven, for all its insistence on grounding, makes deliberate room for the facts you invented. It calls them **undisputed**.

Two kinds of fact, checked two ways

Back in Chapter 1 you learned to tell **borrowed** claims (checkable against the world) from **invented** ones (yours to command). The trust ladder, the sources, the cross-checking — all of Parts II and III — apply only to the borrowed kind. The invented kind needs the opposite treatment: protection **from** fact-checking, not by it.



Borrowed facts are checked against the world; invented facts are checked only against themselves.

> TERM **Undisputed fact**
>
> An **undisputed** (or authorial) fact is one you have marked as your own creative invention — true within your work by decree, with no external truth to check it against. It sits **outside** the trust ladder entirely: it is not a low-rung fact waiting to be grounded, it is an **axiom**. Marking it undisputed tells the tool, "do not question whether this is real — I made it real."

Marking a fact undisputed

In the Facts tree, select an invented fact and press a single key: `u`. It gains a distinct glyph — ※ — and from that moment it is treated as an authorial axiom.

The immediate effect is that `/factcheck` **leaves it alone**. When you audit your corpus (Chapter 8), undisputed facts are excluded from the truth pass — there is nothing to verify — though the audit does **report how many** it skipped, so you always know how much of your world is invented axiom versus borrowed fact. Your

three moons will never be flagged as "unsupported," because support was never the point.

FICTION

This is the feature that makes the Research Assistant safe for a novelist. Mark your world's invented facts undisputed, and the fact-checker stops treating your magic system as a factual error. Your borrowed facts still get the full scrutiny; your invented ones are left to be exactly as you decreed.

NON-FICTION

Non-fiction needs this rarely, but it needs it. A definition you **stipulate** ("in this book, 'early modern' means 1500–1700"), the premise of a thought experiment, an axiom your argument builds on — these are undisputed too: true by authorial declaration, not to be fact-checked.

Coherent, even if invented

An invented fact cannot be checked against the world — but it can still be checked against **itself**. A magic system can contradict its own rules; an invented history can put two decreed events in an impossible order. So there is a separate, optional check made just for authorial facts, asking a completely different question.

```
/undisputed
```

`/undisputed` runs a **common-sense** pass over your undisputed facts — not "is this real?" (it never is; that is the point) but "does this make sense **within its own frame** — is it self-consistent, free of obvious internal contradiction?" Each fact comes back `PLAUSIBLE`, `ODD`, or `INCOHERENT`, and the tree colours its ※ glyph accordingly, so you can see at a glance which of your invented facts hang together and which want a second look.

TERM **Internal coherence**

Internal coherence is consistency **within** an invented frame, judged without reference to the real world. A dragon is not "wrong," but a dragon that is described as both cold-blooded and a source of its own heat may be internally **incoherent**. `/undisputed` checks for that kind of self-contradiction — and, like every check in this book, it only reports; it never rewrites your invention.

IT CHECKS YOUR WORLD; IT NEVER EDITS IT

`/undisputed` will never change a word of an invented fact — it has no standing to. Your creative decisions are yours. It simply holds a mirror up: "within the rules you set, does this hang together?" A `PLAUSIBLE` verdict is reassurance; an `INCOHERENT` one is an invitation to look, not an order to change. And it works in your project's language, so a world written in German or Russian is judged in its own tongue.

The whole picture

Step back and the design resolves into something clean. Your book contains two kinds of fact, and the Research Assistant serves both without confusing them. **Borrowed** facts climb the trust ladder, get cross-checked, and remember their sources — because a reader could look them up. **Invented** facts sit outside the ladder as undisputed axioms, protected from fact-checking, checked only for internal coherence — because you made them, and the only truth they answer to is your own.

A novelist gets a research tool that never mistakes imagination for error. A nonfiction author gets one that respects a stipulated definition as readily as it scrutinises a borrowed claim. Both get the same promise: the tool grounds what should be grounded, protects what you invented, and never edits either.

Which means your corpus is now complete — borrowed facts grounded and checked, invented facts marked and coherent. It is finally time to **use** it: to compose out of everything you gathered, straight into the book you are writing. That is the last working part.

WHAT YOU LEARNED

- Fiction rests on **invented** facts that must be protected **from** fact-checking — there is no external truth to check them against.
- Press `u` in the Facts tree to mark a fact **undisputed** (glyph ※): an authorial axiom that sits **outside** the trust ladder.
- `/factcheck` **excludes** undisputed facts from the truth pass (and reports how many it skipped) — your invented world is never flagged as "unsupported."
- `/undisputed` checks invented facts for **internal coherence** — self-consistency within their own frame (`PLAUSIBLE` / `ODD` / `INCOHERENT`) — in the project language, and never rewrites them.
- The design resolves cleanly: borrowed facts are grounded and checked; invented facts are marked and kept coherent; the tool never confuses the two, and never edits either.

PART VI

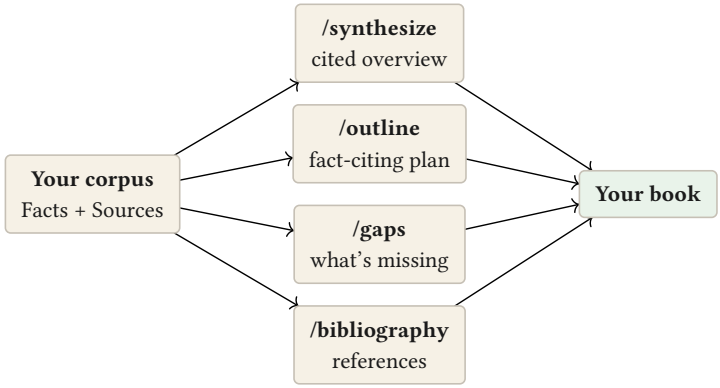
Composing Out

CHAPTER 12

12

Composing From What You Know

Everything in this book so far has **built** something: a corpus of facts, grounded, cross-checked, and kept honest. But a corpus is not a book. At some point the research has to turn back into writing — and this is where most tools abandon you. You have the reading; now go and use it, somehow, in the other window. Inkhaven does not abandon you here. The corpus you built is **composable**: it can produce a cited overview, a working outline, and a list of what you still don't know — drawn entirely from the facts you verified.



The corpus you built is not the end product — it is the raw material the last commands turn back into your book.

There is one rule that governs all three commands in this chapter, and it is what makes them trustworthy: they compose from **your corpus only**. They do not go back to the model's free imagination; they work with the facts you established and checked, and they cite each one. This is the difference between "write me something about aqueducts" and "assemble what **I** have verified about aqueducts, and show your sources."

A grounded overview: `/synthesize`

Point `/synthesize` at a topic, and it retrieves the facts you have kept that bear on it and weaves them into a coherent overview — citing each claim back to the fact it came from, and **flagging where your corpus is thin**.

```
/synthesize the Roman water supply
```

> **TERM Synthesis**
>
> A **synthesis** draws together many separate facts into one coherent account of a topic. Here it is **grounded**: built only from facts you have verified, with each claim cited to its source, and honest about the gaps. It is not the model telling you about the topic — it is the model organising what **you** established about it.

The honesty about gaps matters as much as the overview itself. A synthesis that says "the corpus establishes the capacity and the route, but is silent on the construction dates" is telling you exactly where more research is needed before you write. It is a mirror held up to your own reading.

The research-to-writing bridge: `/outline`

`/outline` takes the same grounded material and shapes it differently — into a **structured outline** for actually writing about the topic, where each point cites the facts that support it, and anything your corpus does **not** cover is marked plainly as `(needs research)`.

> TERM **The research-to-writing bridge**
>
> The gap every writer knows: you have done the reading, and now face the blank page. An `/outline` is the plank across that gap — it turns your verified corpus into a shape you can write **into**, with the supporting fact sitting under each point and the holes marked. You are no longer starting from nothing; you are starting from what you know, arranged.

You copy the outline into your manuscript and write from it — and because each point is backed by a cited fact, you write **grounded**, without stopping to re-check. The `(needs research)` marks become your to-do list for the gaps.

What you don't know yet: `/gaps`

The most useful thing a corpus can tell you is sometimes what is **missing** from it. `/gaps` asks exactly that: given everything you have on a topic, what are the open questions your corpus cannot yet answer?

```
/gaps the Roman water supply
```

It returns a list of specific, concrete questions — "you have the aqueduct's capacity but not its construction date; you have the route but not the gradient" — each one a piece of research you now know to go and do. And these questions are not a dead end: in the next part you will see them become the **input** to headless research, so the corpus can be told to go and fill its own gaps.

FICTION

Before you write a scene set in a place, `/synthesize` everything you have established about it into a brief, `/outline` the scene grounded in those facts, and let `/gaps` tell you the one detail you still need to look up. You draft from a foundation instead of improvising and hoping it holds.

NON-FICTION

`/synthesize` produces a cited summary of the literature you have gathered — the backbone of a review section. `/outline` turns your sources into the skeleton of an argument, each claim already footnoted. `/gaps` is your literature-gap analysis, done from your own corpus.

IT COMPOSES; YOU WRITE

These commands never write into your manuscript on their own, and they never invent to fill a hole — a gap is reported as a gap, not papered over. What they produce is a cited draft **of the research**, for you to lift, edit, and make your own. The words that end up in your book are still yours; the tool just makes sure you start from what you actually know.

The corpus pays off

This is the moment the whole discipline pays for itself. Every fact you grounded, every source you cross-checked, every citation that filed itself — it was all so that, at **this** point, you could ask your own knowledge base to organise itself into something you can write from, honest about its own limits, cited throughout. The research was never the goal; this is.

One product of the corpus deserves its own chapter, because it is the one a non-fiction author often has to **ship**: the bibliography. All those citations that quietly filed themselves as you researched are about to become a formatted reference list, with a single command. That is next.

WHAT YOU LEARNED

- The corpus is **composable**: `/synthesize`, `/outline`, and `/gaps` turn your verified facts back into writing — drawing on your corpus **only**, and citing each claim.
- `/synthesize <topic>` weaves your kept facts into a **grounded, cited overview** and flags where the corpus is thin.
- `/outline <topic>` is the **research-to-writing bridge**: a fact-citing outline you write into, with holes marked `(needs research)`.
- `/gaps <topic>` lists the **open questions** your corpus can't answer yet — which the next part turns into fresh research.
- These commands compose but never write your prose or invent to fill gaps — the payoff of all the grounding is a cited draft you make your own.

CHAPTER 13

13

The Bibliography That Built Itself

Somewhere back in Chapter 5, a small promise was made and then left to run quietly in the background. Every time you kept a fact grounded on a scholarly paper — every `/openalex` and `/arxiv` result you turned into a `/fact` — a proper citation was filed into your **Sources** book without you doing anything. Every reference you imported went there too. All that time, a bibliography has been assembling itself out of the provenance you accumulated. This chapter is where you collect on it.

Where the citations went

Recall the third of your research books, introduced back in Part I. **Facts** holds what you trust; **Notes** holds what you're unsure of; **Sources** holds your bibliography — the citable works behind your facts. You have barely had to touch it, and yet it has been filling the whole time.

> TERM **Bibliography**
>
> A **bibliography** is the list of sources a work draws on — its references. In a non-fiction book it is often part of the deliverable itself; in fiction it may be a private record or a "further reading" note. Either way, a hand-maintained bibliography is tedious and error-prone. The point of this chapter is that yours was never hand-maintained: it was a by-product of researching honestly.

Collecting it: `/bibliography`

One command turns the accumulated Sources book into a formatted reference list:

```
/bibliography
```

It walks the citations you gathered and emits them as **BibTeX** — the standard, portable citation format — ready to copy into a references section, or into the tools that typeset one.

> TERM **BibTeX**
>
> **BibTeX** is a plain-text format for bibliographic entries — author, title, year, identifier, one block per source. It is the lingua franca of citations: nearly every academic tool reads it, and Inkhaven's own book-assembly can render it into a formatted reference list. It is what `/bibliography` produces.

Because each entry was filed from a real source with a real identifier — a DOI, a catalogue number — the list is not a guess at how a citation **should** look; it is the actual, resolvable reference. You copy it with a keystroke, or, if you would rather not open the tool at all:

```
inkhaven research --bibliography --out references.bib
```

That writes the whole bibliography to a file from the command line — the same citations, without the interface. Later parts return to this **headless** way of working; here it is enough to know that your reference list is one command away whether you are inside Inkhaven or scripting around it.

FICTION

Fiction rarely ships a bibliography, but the research behind a serious novel deserves a record. `/bibliography` gives you a "sources consulted" list for an author's note, an acknowledgements page, or simply your own archive of what grounded the world — assembled for free as you worked.

NON-FICTION

This is often part of what you are **delivering**. The references section that a reviewer or an editor expects builds itself from your provenance: every claim you grounded on a paper is already a citation, and `/bibliography` collects them into the list your manuscript needs. No separate citation manager, no retyping, no drift between what you cited and what you listed.

THE QUIET REWARD OF HONESTY

There is a lesson hiding in this chapter. The bibliography assembled itself only because, all along, every fact recorded **where it came from**. Provenance — the small, almost invisible habit from Chapter 1 — is what makes a free bibliography possible. The discipline of grounding was never just about being right; it was about building a corpus that could give this much back.

Managing the Sources book from the shell

`/bibliography` is the quick path; the Sources book is also a first-class thing you can manage directly, with a dedicated `inkhaven sources` command, for working with reference managers like Zotero. It reads and writes the same Sources book, from outside the Research screen:

```
inkhaven sources list
inkhaven sources export --format csl-json --out sources.json
inkhaven sources import zotero-export.bib
inkhaven sources check
```

`export` writes your citations out in one of two formats: **BibTeX**, as `/bibliography` does, or **CSL-JSON** — the format Zotero, Mendeley, and the wider citation ecosystem read and write. Because Inkhaven also **imports** BibTeX, the round-trip is complete: a library curated in Zotero comes in, your research adds to it, and it goes back out. And `sources check` validates the book — flagging a missing key or a malformed entry, and exiting non-zero — so you can wire it into a continuous-integration step and never ship a broken reference.

> TERM **CSL-JSON**
>
> **CSL-JSON** is the JSON citation format of the Citation Style Language ecosystem — what Zotero and most modern reference managers speak natively. Where BibTeX is the LaTeX world's lingua franca, CSL-JSON is the interchange format between citation tools. `sources export --format csl-json` closes the round-trip with them.

The loop closes

Step back to the project arc you met in Chapter 2 — acquire, cross-check, maintain, compose. You have now travelled the whole circle. You **acquired** facts from every rung of the ladder; you **cross-checked** the load-bearing ones and audited the rest; you **maintained** the corpus by upgrading guesses and flagging stale facts; and in this part you **composed** it back out — a cited synthesis, a working outline, a list of what's missing, and a bibliography that built itself. The knowledge base that began as an empty tree has become something that writes back into your book.

Two things remain. Everything so far has been hands-on, one command at a time — but some research is better done in bulk, headlessly, while you do something else; that is the next part. And then a single worked example, start to finish, to see the whole workflow move as one. The tools are all in your hands now; what's left is learning to run them at scale and in concert.

WHAT YOU LEARNED

- Your **Sources** book filled itself the whole time: every scholarly `/fact` and every `/import` filed a citation automatically.
- `/bibliography` collects those into **BibTeX** — the portable, resolvable citation format — ready to copy or typeset.
- `inkhaven research --bibliography --out file.bib` writes the same list from the command line, no interface needed.
- The free bibliography is the reward of **provenance** — the habit of every fact recording its origin, from Chapter 1, paying off.
- With this the project arc closes — acquire, cross-check, maintain, **compose** — and the corpus writes back into your book.

PART VII

Working at Scale

CHAPTER 14

14

Research at Scale

Everything so far has been hands-on: you ask, you read, you confirm, one fact at a time. That is exactly right when you are thinking — research is a conversation, and a conversation wants a person in it. But some research is not thinking; it is **fetching**. A list of forty questions you already know you need answered. A folder of PDFs to bring in. A book to ingest. For that kind of work you do not want to sit and confirm forty times — you want to set it running and come back to a report. Inkhaven can do all of this **headlessly**, from the command line, with no interface at all.

> **TERM Headless**
>
> A tool runs **headless** when it works without its interactive interface — driven from the command line, unattended, its results written to a file. The same Research Assistant you have been using in its two-pane screen can also be invoked as `inkhaven research …` with flags, to do in bulk and in the background what you would otherwise do by hand.

Answering a list: `--batch`

The headline is `--batch`. Give it a file of questions — one per line — and it researches each one, distils a candidate fact, scores its own confidence, and writes a report:

```
inkhaven research --batch questions.txt --out findings.md
```

By default it is conservative: it **proposes** facts and reports them, but leaves the confirming to you — the gate is still yours, just deferred to when you read the report. If you want it to actually insert the high-confidence findings unattended, you say so explicitly, and set the bar:

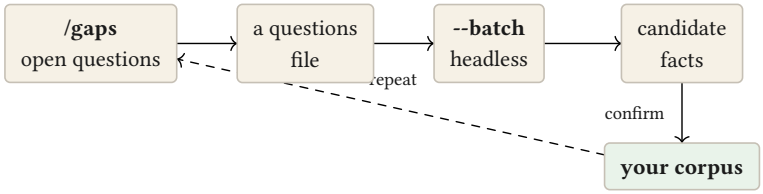
```
inkhaven research --batch questions.txt --auto-confirm --
confidence 0.8
```

Now facts whose confidence clears the threshold are kept automatically, and the rest are reported for your judgement. The threshold is the dial between "do it all for me" and "show me your work" — and it is opt-in, because unattended insertion is exactly the kind of thing that should never happen by surprise.

> **TERM Batch research**
>
> **Batch research** answers a whole list of questions in one unattended run, writing a report of what it found and how confident it was. It trades the interactive gate for a confidence threshold you set — a way to do the fetching kind of research at volume while keeping the thinking kind for the screen.

Closing the loop from `/gaps`

Remember where a list of questions comes from. In Part VI, `/gaps` handed you exactly that — the open questions your corpus could not yet answer. Feed those questions to `--batch`, and the loop closes: the corpus tells you what it is missing, you point a batch run at the holes, and it comes back with candidate facts to fill them.



The corpus can be told to go and fill its own gaps while you do something else.

This is the corpus filling its own gaps. You `/gaps` a topic, drop the questions into a file, run `--batch` over them while you do something else, and return to a report of proposed facts — each one still crossing your judgement before it is kept. Research that used to be a day of tab-juggling becomes a run you start and walk away from.

Bringing material in: `--import` and `--sync`

The other headless jobs are about **material** rather than answers.

`--import <path>` ingests a document — a PDF, a text file, a whole folder — into your corpus from the command line, the same ingestion you met interactively, now scriptable.

`--sync <folder>` goes one step further: it **registers** a folder so that whenever its contents change, the material is re-imported automatically. Point it at the directory where your reading accumulates, and your corpus stays current with your reading without a manual step.

```
inkhaven research --import ~/reading/aqueducts.pdf
inkhaven research --sync ~/reading/
```

And `--gutenberg` brings a public-domain book in headlessly, the command-line twin of the `/gutenberg` you already know.

FICTION

Batch-research the texture questions for a whole act at once — "what did X look like, sound like, cost?" — overnight, and skim the report in the morning for the details worth keeping. `--sync` a folder of period sources so anything you drop in is searchable by the time you sit down to write.

NON-FICTION

This is where a large project becomes tractable. Turn a literature-gap list into a batch run; `--sync` the folder where your references land so the corpus never drifts from your reading; script `--bibliography` into your build so the references section regenerates on every draft. The whole research apparatus becomes part of your pipeline.

AUTOMATION WITH THE SAFETY KEPT

Headless mode relaxes the **interactive** gate, but not the **principle**. By default nothing is inserted without your review; unattended insertion is opt-in and threshold-gated; provenance is still recorded on every fact; and your manuscript is never touched. Scale changes how much you can do at once — it does not change who decides what is true.

The tools are all in your hands

You now have the entire workflow, at every scale: one fact at a time in the interactive screen, and whole lists and folders at once from the command line. Acquire, cross-check, maintain, compose — by hand when you are thinking, headless when you are fetching. There is nothing left to introduce.

What is left is to see it all **move together**. The final chapter is a single worked example — one novelist and one non-fiction author, each taking a real claim from a first question to a grounded, cited fact and on into their book — so the pieces you have met one at a time run, at last, as one workflow.

WHAT YOU LEARNED

- Interactive research is for **thinking**; **headless** research (`inkhaven research …`) is for **fetching** — bulk, unattended, written to a report.
- `--batch <file>` answers a list of questions; it proposes by default, and only inserts unattended with `--auto-confirm` above a `--confidence` threshold you set.
- `/gaps` → a questions file → `--batch` closes the loop: the corpus fills its own holes while you do something else, every fact still crossing your judgement.
- `--import` ingests material from the CLI; `--sync` keeps a folder's contents auto-imported; `--gutenberg` and `--bibliography` have headless twins too.
- Scale relaxes the interactive gate but keeps the principle: review by default, provenance always, prose never touched — you still decide what is true.

CHAPTER 15

15

Research That Researches Itself

`--batch` answered a list of questions you **wrote**. But the hardest part of research is often not answering the questions — it is knowing which questions to ask. You start with a topic and a vague sense that there is more you don't know than you do. This chapter is about the tools that close that gap: research that plans its own questions and follows its own leads. Two of them — an autonomous research **loop**, and citation **snowballing** — and one principle that governs both: they fill your fact base and your sources, but they never decide what you trust.

`--agentic` — research that plans itself

Give the assistant a **topic** rather than a question, and it does what a diligent researcher does: breaks the topic into the specific questions it raises, answers each one against your sources, writes down what it finds, and then looks at what it has and asks what's still missing — and keeps going until the topic is covered or its budget is spent.

```
inkhaven research --agentic "the 1918 pandemic" --out flu.md
```

Watch a run unfold. Each **round** plans a batch of sub-questions, researches them, and emits a Fact for each; then a critic proposes follow-ups for the gaps:

● ● ● *inkhaven research --agentic — a run in progress*

```
» agentic round 1: 5 sub-question(s)
· What caused the 1918 influenza pandemic?
· How many people did it kill worldwide?
· Why was it called the "Spanish" flu?
· Which age groups were most affected?
· How did the pandemic end?
! contradiction: "~50 million dead" ⇄ "17–100 million,
  a contested range"
» agentic round 2: 2 sub-question(s)
· What explains the wide range in the death-toll estimates?
· Why did it kill healthy young adults so often?
✓ 7 Fact(s) over 2 round(s), converged · ! 1 contradiction
  — resolve before trusting (untrusted, model provenance).
  Review in the Facts book: promote, dispute, /factcheck.
```

Three things just happened that are the whole point of the feature. It **wrote its own questions** — you never listed them. It **noticed a contradiction** between two facts it had gathered and, in the next round, asked a question aimed squarely at resolving it. And it **stopped on its own** — not after a fixed number of steps, but when the critic had nothing left to add.

The output is your Facts book, not an article

This is the part that makes agentic research trustworthy rather than alarming. It does **not** hand you an essay to reconcile against what you already believe. Every finding is written as a Fact — a paragraph in your Facts book — carrying its provenance and marked **untrusted**, exactly as if you had researched it by hand and not yet confirmed it. The report is a table of contents into what it wrote:

● ● ● *flu.md — the run's report*

```
# Agentic research report

**Topic:** the 1918 influenza pandemic

7 sub-question(s) · 2 round(s) · 7 Fact(s) emitted into
the Facts book (untrusted — review) · stopped: converged.

## ! Contradictions among the emitted facts (1)

- **~50 million died worldwide** ⇄ **estimates range
  from 17 to 100 million** — one states a settled figure,
  the other frames it as contested

## 1. What caused the 1918 influenza pandemic?

**Cause of the 1918 pandemic**
An H1N1 influenza A virus of probable avian origin…
_confidence 0.86 · inserted → facts/cause-1918_
…
```

So an agentic run leaves you exactly where a good afternoon in the library would: with a stack of sourced notes, some of which disagree, none of which you have decided to believe yet. The trust ladder (Chapter 8) and everything on it — fact-checking, refutation, the undisputed mark — apply to these facts natively, because they **are** ordinary facts. Nothing about the pipeline is bypassed; the research just got faster.

> **WHY IT FLAGS ITS OWN CONTRADICTIONS**
>
> The risk of any tool that writes facts automatically is that it quietly fills your base with claims that disagree with each other. So after each round the agentic loop runs the same `⇄` contradiction check you'd run by hand (Chapter 8) over **its own** emitted facts, feeds any clash back to the critic to resolve, and flags what remains at the top of the report. Autonomous emission is only safe because the conflicts it introduces are surfaced, not hidden — review is targeted, not a blind audit.

Where it stops, and the switch that turns it off

An autonomous loop must never run away. This one stops at the first of three bounds, and the run always tells you which: it **converges** (the critic finds no more gaps), it hits the **round cap**, or it spends its **question budget** — a hard ceiling on how many facts one run can emit, and therefore on its cost. Sub-questions dropped for want of budget are logged, never silently cut.

All three are yours to set, and the whole behaviour is a switch — on by default, off whenever you want it off. In your project's `inkhaven.hjson`:

```
research: {
  agentic: {
    enabled: true          // false disables agentic runs
    max_subquestions: 6    // total Facts budget per run
    max_rounds: 3          // iterate rounds (1 = one pass)
  }
}
```

With `enabled: false` the command simply refuses with a hint; your ordinary, hands-on research is untouched. Autonomy is a tool you pick up, never one that is running whether you asked for it or not.

`--snowball` — following the citations

The other way research grows is not by asking more questions but by following the **sources** you already have. One good paper cites a dozen others and is cited by a hundred more; that neighborhood is where the rest of the literature lives. Snowballing walks it for you. Give it a seed — a title, a DOI, a topic — and it follows the citation graph both ways:

```
inkhaven research --snowball "attention is all you need"
```

● ● ● *inkhaven research --snowball — the citation neighborhood*

```
· seed: Attention Is All You Need  (openalex:W2626778328)

## References (backward) — 10 works the seed cites
- Deep Residual Learning for Image Recognition
    — He et al., 2016    openalex:W2194775991
- Effective Approaches to Attention-based Neural MT
    — Luong et al., 2015    openalex:W1902237438
- Generating Sequences With Recurrent Neural Nets…

## Citations (forward) — 10 works that cite the seed
- An Image is Worth 16x16 Words (Vision Transformer)
    — Dosovitskiy et al., 2020    openalex:W3094502228
- DistilBERT, a distilled version of BERT — Sanh, 2019
- Highly accurate protein structure prediction with
    AlphaFold — Jumper et al., 2021    openalex:W3177…
```

Backward are the works your seed stands on; **forward** are the works that stand on it, most-cited first — often the reviews and landmark follow-ups that tell you where a line of research went. In one command you've turned a single paper into a map of its scholarly surroundings.

It surfaces, it doesn't swamp

Notice what snowball did **not** do: it did not drag thirty papers into your Sources book. It reported them, each with its identifier, and left the choosing to you — because a research corpus is only useful if it's **curated**, and thirty auto-ingested papers of uneven relevance is not a corpus, it's a mess. Bring in the ones that matter with `/openalex` (Chapter 5); the rest you now simply know exist. Same discipline as the agentic loop: the machine widens your reach, you decide what enters.

WHAT YOU LEARNED

- `--agentic "<topic>"` researches a topic **autonomously**: it plans its own sub-questions, answers each, and **emits the findings as untrusted Facts into your Facts book** — never a standalone article — so the whole trust ladder applies to them.
- It **iterates** — a critic proposes follow-ups for the gaps — and stops on the first of three bounds it reports: converged, round cap, or question budget. It never runs away.
- After each round it runs the `⇄` contradiction check over its **own** facts, feeds clashes back to the critic, and flags what remains — so autonomous emission stays trustworthy and review is targeted.
- The whole behaviour is `research.agentic` in HJSON — **on by default**, and switched off with `enabled: false`.
- `--snowball "<seed>"` follows a paper's citations **backward** (its references) and **forward** (its citers) on OpenAlex, reporting the neighborhood for you to ingest selectively — widening your sources without swamping them.

PART VIII

A Complete Walkthrough

CHAPTER 16

16

One Fact, End to End

You have met every tool in this book one at a time. This chapter runs them as one workflow. We will follow a single subject — a Roman aqueduct — from a first question to a grounded, cross-checked, cited fact, and on into a piece of writing. Two authors make the journey side by side: a **novelist** who needs the aqueduct to feel real in a scene, and a **historian** who needs a claim about it to be defensible in an argument. Same tools, same order; two destinations.

Stage 1 — Ask

Open the Research Assistant, start a thread for the topic, and ask a plain question.

```
How much water did a major Roman aqueduct deliver per day?
```

The answer arrives grounded on whatever you have already kept — which, on a new project, is not much, so it leans on the model. It gives you a figure. That figure is currently a guess: the bottom rung.

FICTION

You need the **scale** to feel right in a crowd scene — enough to sense the city's thirst, not a number you will print.

NON-FICTION

You need the **figure itself**, with evidence, because it anchors a paragraph of your argument.

Stage 2 — Climb to a real source

Do not keep the guess. Go up the ladder. The aqueduct is a real thing, so try the structured and scholarly rungs:

```
/wikidata Aqua Claudia
/openalex Roman aqueduct hydraulic capacity
```

Wikidata hands you the identified entity and its properties; OpenAlex hands you a paper with a daily-capacity figure and a DOI. Now you have a claim that starts far higher than the model's guess — and the OpenAlex result, when you keep from it, will file its own citation into your Sources book.

Stage 3 — Keep it

Take the fact, and confirm it at the gate.

```
/fact Aqua Claudia delivered roughly 190,000 cubic metres of
water per day
```

Because you grounded on a scholarly source, the fact lands cited — provenance `openalex`, DOI attached — and its citation is now in Sources without another keystroke.

Stage 4 — Cross-check the load-bearing claim

This figure is going to carry weight, so do not trust a single source. Triangulate it:

```
/triangulate Aqua Claudia delivered about 190,000 cubic metres
per day
```

The Assistant queries the structured and scholarly sources at once and reports the agreement. Suppose it comes back with one clear `SUPPORTS` and no `CONTRADICTS` — the claim is corroborated across independent references, and you keep it knowing it was tested from more than one side.

> **FICTION**
>
> For the novelist, this is where you stop. The number will never appear on the page; you needed to know the **order of magnitude** is real so the scene's sense of scale is honest. One check is plenty.

> **NON-FICTION**
>
> For the historian, one supporting source is a beginning. You might also run `/upgrade` on any related guesses, and turn the refutation gate on so a determined sceptic pass runs before each new claim commits. The figure is going in a footnoted paragraph; it should survive attack as well as find support.

Stage 5 — Audit, before anyone else does

Chapters later, your corpus has grown. Before a beta read or a submission, sweep it:

```
/factcheck
```

The tree lights up with verdict glyphs. A ✕ appears on a travel-time fact you kept early — it contradicts the aqueduct's dates. You select it and ask `/whatswrong`, which explains the clash, and you fix the wording. The contradiction that a sharp reader — or a referee — would have caught, you caught first.

Stage 6 — Compose it back out

Now use what you built. Ask the corpus to organise itself:

```
/synthesize the water supply of imperial Rome
/gaps the water supply of imperial Rome
```

`/synthesize` returns a cited overview, drawn only from your verified facts, honest about where it is thin. `/gaps` lists what is still missing — and those questions go into a file you hand to a headless `--batch` run overnight, so the holes fill themselves while you sleep.

FICTION

`/outline` the scene grounded in your facts, copy it into the manuscript, and write — the aqueduct's scale, the city's thirst, the engineering, all resting on ground you verified, none of it slowing the prose.

NON-FICTION

`/outline` the section, each point already citing its source; write the argument; then `/bibliography` collects every citation you accrued into a references list — the deliverable assembling itself from the provenance you kept all along.

What just happened

Trace the arc. You **acquired** a fact, climbing from a guess to a cited scholarly claim. You **cross-checked** it against independent sources. You **maintained** the corpus, catching a contradiction in an audit. And you **composed** it back out — a synthesis, an outline, a bibliography — into the book you are writing. Every fact you kept remembered where it came from; nothing was fact-checked that you invented; your prose was never touched by the tool.

That is the whole discipline, and it is not really about commands. It is about a single habit made effortless: **never let a borrowed claim rest on nothing, and always remember where it came from.** Do that, and a reader can trust the facts you borrowed — which is what lets them believe the world you invented.

The appendices that follow are for reference: every command in one place, the provenance rungs and their glyphs, and a glossary of the terms this book defined. Keep them at your elbow. The workflow you now have in your hands is small, honest, and yours.

WHAT YOU LEARNED

- The whole workflow in order: **ask → climb to a real source → keep → triangulate → audit → compose**.
- A novelist and a non-fiction author walk the same path to different ends — a world that **feels** solid, and an argument that **is** defensible.
- Grounding is not about commands; it is one habit made effortless: never let a borrowed claim rest on nothing, and always record where it came from.
- Because the borrowed facts can be trusted, the reader can believe the invented ones — which is the entire point.

APPENDIX A

A

Command Reference

Every command this book taught, in one place, grouped by the part of the workflow it belongs to. In the Research Assistant, type a single `/` to open the searchable command palette — you never have to memorise these; this list is for browsing.

Asking and keeping

- **(a plain question)** — ask in ordinary language; the answer is grounded on the facts you have already kept.
- `/fact <claim>` — keep a claim as a trusted **Fact**; crosses the confirmation gate.
- `/note <claim>` — keep a speculative claim as a **Note** instead.

- `/verify` — probe the model's confidence in its last answer before you keep it, so a shaky reply doesn't become a Fact.
- `/diff <claim>` — check a claim against what you have already kept and warn if it near-duplicates an existing fact (tuned by `research.dedup_warn_score`).
- `/promote` — promote a selected Note into the Facts book.
- `u` **(key, in the Facts tree)** — toggle the selected fact **undisputed** (※): an authorial axiom, exempt from fact-checking.

Authoritative sources

- `/web <query>` — search and ground a **cited** answer on real pages; a `/fact` from it is fact-checked at the gate.
- `/web --ingest <query>` — embed the fetched pages into your corpus as searchable material instead of answering.
- `/wikidata <query>` — structured facts by **Q-id**; gate-skipped.
- `/geonames <query>` — real places from the GeoNames gazetteer; gate-skipped. Needs a free `research.geonames.username`.
- `/openalex <query>` — the top scholarly work (DOI); auto-files its citation to Sources.
- `/arxiv <query>` — the top preprint; auto-files its citation.
- `/gutenberg <query>` **(alias `/pg`)** — ingest a public-domain book; `<PG#>` selects an exact edition, `--chapter N` a single chapter.

Cross-checking and maintenance

- `/triangulate <claim>` — cross-check a claim against the structured and scholarly sources at once; reports SUPPORTS / CONTRADICTS / SILENT.
- `/factcheck` — audit the whole Facts book for per-fact truth and cross-fact consistency; marks each fact with a verdict glyph (✓ / ? / ✕).
- `/whatswrong` — explain why the selected flagged fact failed, and what the correct information appears to be.
- `/upgrade [facts/path]` — re-ground a `model` fact on a corroborating source and raise its provenance tier in place (the wording is never changed).
- `/stale [days]` — list `model` / `web` facts older than **days** (default 90) for re-verification.
- `/undisputed` — check your undisputed (authorial) facts for **internal coherence** (PLAUSIBLE / ODD / INCOHERENT), in the project language.

- `/deadsources` — scan your kept web sources for link-rot and flag the ones that no longer resolve, so a citation does not quietly die under you.
- `/sources` — list every fact's provenance in one view — the interactive companion to the provenance ladder.
- `/forget <source>` — remove an imported source and the material it brought in.

Computing

- `/calc <expr>` — compute a fact: unit conversions, great-circle distances, compound growth, list reductions, and domain formulas (geography, astronomy, climate, economy). Provenance `computed`.
- `/world` — browse the project's World-simulation facts; `/calc` can read them.
- `/rag <mode>` — choose what an ordinary question is grounded on: `facts+full` (both, the default), `facts` (your kept facts only), or `full` (the whole corpus).
- `/chain <q1 → q2 → q3>` — run a sequence of questions as a pipeline, each building on the last, for a multi-step line of research.

Composing out

- `/synthesize <topic>` — a grounded, cited overview drawn only from your kept facts, honest about where the corpus is thin.
- `/outline <topic>` — a fact-citing outline to write into; uncovered points marked `(needs research)`.
- `/gaps <topic>` — the open questions your corpus cannot yet answer.
- `/bibliography` — collect the Sources book's citations into BibTeX.

Citations — the Sources book

The Research Assistant files a citation to the **Sources** system book every time it grounds an answer on a scholarly work or a page. A separate `inkhaven sources` command manages that book from the shell, for interchange with reference managers.

- `inkhaven sources list` — list every citation in the Sources book.

- `inkhaven sources check` — validate the entries (missing keys, malformed fields); exits non-zero on a problem, so it fits a CI step.
- `inkhaven sources import <file.bib>` — bring citations in from a BibTeX file (e.g. exported from Zotero).
- `inkhaven sources export --format bibtex|csl-json [--out <file>]` — write the citations out. **CSL-JSON** closes the round-trip with Zotero and other citation managers; BibTeX suits LaTeX.

Headless (command line)

- `inkhaven research --batch <file> [--out report.md]` — research a list of questions unattended; proposes by default.
- `… --auto-confirm --confidence <0..1>` — insert findings above the confidence threshold automatically; report the rest.
- `inkhaven research --import <path>` — ingest a document or folder from the command line.
- `inkhaven research --sync <folder>` — register a folder for re-import whenever its contents change.
- `inkhaven research --gutenberg "<query|PG#>"` — ingest a public-domain book headlessly (accepts a leading `--chapter N`).
- `inkhaven research --bibliography [--out refs.bib]` — write the bibliography to a file, or to standard output.
- `… --thread <name>` — open (or, headless, act on) a named research thread; `--list-threads` shows them, `--export-thread <name> --format <fmt> --out <file>` writes one out.

Window and navigation

Housekeeping inside the Research screen — none of it changes your corpus:

- `/goto <path>` — jump the Facts tree to a book, chapter, or fact by path.
- `/save` — save the current thread; `/clear` — clear the conversation window (your kept facts are untouched).

APPENDIX B

B

The Trust Ladder and Provenance

Every fact you keep records a **provenance** — where it came from — and that origin places it on the trust ladder. This appendix lists the origins from firmest to softest, with the glyph each wears in the Facts tree and whether it crosses the fact-check gate.

The rungs, top to bottom

≡	**computed**	A fact you derived by calculation (/calc). The firmest rung — anyone can re-run it. *Gate-skipped.*

≡	**simulation**	A deterministic fact from the World book's simulation. As firm as computed, for the same reason. *Gate-skipped.*
◆	**wikidata**	A structured datum, cited by Q-id. *Gate-skipped.*
⊕	**geonames**	A real place from the gazetteer, cited by id. *Gate-skipped.*
§	**openalex**	A scholarly work, cited by DOI; auto-filed to Sources. *Gate-skipped.*
§	**arxiv**	A preprint, cited by id; auto-filed to Sources. *Gate-skipped.*
■	**document**	Drawn from a source you imported into your corpus. *Fact-checked at the gate.*
↑	**promoted**	A Note you promoted into the Facts book. *As its underlying source.*
◇	**web**	Grounded on a cited web page. *Fact-checked at the gate.*
·	**model**	The model's unaided answer — an educated guess. *Fact-checked (and refuted, if enabled).*

TWO GLYPHS OUTSIDE THE LADDER

Two marks are not rungs at all. The verdict glyphs from an audit — ✓ passed, **?** dubious, ✕ failed (`/factcheck`) — sit **on top of** a fact's tier glyph to report its last check. And ※ marks an **undisputed** (authorial) fact, which sits outside the ladder entirely: it is exempt from `/factcheck` and checked only for internal coherence by `/undisputed` . Its ※ takes on the coherence verdict's colour — plausible, odd, or incoherent.

Reading a fact's provenance

The tier glyph is the at-a-glance version. The full record — the specific source, the query that produced it, any check verdict folded in — travels with the fact and answers the one question this whole book is built around: **how do you know?**

The point of the ladder is not to forbid the low rungs. A novelist grounding the feel of a place on a web page has done legitimate work; the ◇ simply keeps that fact honest about where it stands. What the ladder insists on is only this — that a fact never pretend to be firmer than its origin, and that **you always know which rung you are standing on.**

APPENDIX C

C

Glossary

Every term this book defined, gathered in one place. Each was introduced in a marked box the first time it was needed; here they are, alphabetical, for looking up.

Audit
A pass over your whole Facts book at once (`/factcheck`), rather than a check of a single claim. It catches old facts never re-examined, and contradictions between facts that are each fine alone.

Batch research
Answering a whole list of questions in one unattended run (`--batch`), writing a report of what was found and how confident it was. It trades the interactive gate for a confidence threshold you set.

Bibliography
The list of sources a work draws on — its references. Yours is never hand-maintained: it accumulates in the Sources book as a by-product of grounding, and `/bibliography` collects it.

BibTeX
A plain-text format for bibliographic entries — author, title, year, identifier — read by nearly every citation tool. What `/bibliography` produces.

Confirmation gate
The mandatory pause between the Assistant proposing a fact and the fact entering your book. You edit, accept, or discard; nothing reaches your Facts without it. The Assistant proposes; you dispose.

Corpus
The whole body of research material gathered for a project — Facts, Notes, Sources, and imported documents. It grows as you work, and you can search and compose from it.

CSL-JSON
The JSON citation format of the Citation Style Language ecosystem — what Zotero and most modern reference managers read and write. Where BibTeX serves LaTeX, CSL-JSON is the interchange format between citation tools; `inkhaven sources export --format csl-json` produces it.

Deterministic
Following from inputs by a fixed rule, so anyone who runs the same computation gets the same answer. Deterministic facts (computed, simulated) are the firmest rung — you trust arithmetic, not a source.

DOI
A permanent code pointing to a specific scholarly work — the scholarly equivalent of a Q-id. A claim cited by DOI can be resolved to the exact paper by any reader.

Fact
A specific, checkable claim your writing rests on — a date, number, name, cause — the opposite of a thing you invented. Grounding the checkable kind is the craft of this book.

Fact-check gate
For a fact drawn from the web or the model, a single-claim accuracy check the gate runs before the fact commits. ACCURATE passes; DUBIOUS / INACCURATE shows its reasoning and asks you to confirm again. It informs; it never blocks.

Gazetteer
A geographical dictionary: a database of place names with their locations, regions, feature types, and populations. `/geonames` reads one.

Grounding
Connecting a claim to something outside your own head that supports it — a source, a computation, a cross-checked agreement. A grounded claim can answer "how do you know?"

Grounding on your corpus
Before answering a question, the Assistant retrieves the most relevant facts you have already kept and hands them to the model as context — so answers stay consistent with what you established, and improve as your corpus grows.

Headless
Working without the interactive interface — driven from the command line, unattended, results written to a file. `inkhaven research …` with flags.

Ingesting a source
Bringing a source's full text into your corpus, split into searchable passages, so the Assistant can retrieve the relevant ones and quote the actual text rather than a summary.

Internal coherence
Consistency within an invented frame, judged without reference to the real world. `/undisputed` checks authorial facts for self-contradiction; it never rewrites them.

Preprint
A scholarly paper shared publicly before formal peer review — fast and current, but less settled than a published article. arXiv is the largest preprint server.

Provenance
The recorded origin of a fact — its rung on the trust ladder and the specific source. It travels with the fact silently and answers "how do you know?" any time.

Q-id
A stable Wikidata code (like `Q220` for Rome) that never changes, so a fact cited by Q-id can be re-checked years later.

Refutation
A check that actively tries to disprove a claim — the mirror of triangulation. A claim that survives a genuine attempt to break it has earned more trust than one merely unchallenged.

Research-to-writing bridge
The plank across the gap between having done the reading and facing the blank page. An `/outline` turns your verified corpus into a shape you can write into, each point backed by a cited fact.

Staleness
When enough time has passed that a fact's grounding may no longer hold. `/stale` surfaces the soft, aging facts (model / web) for a second look.

Structured data
Information stored as discrete, labelled facts — subject, property, value — each with a stable identifier, rather than free-flowing prose. Nothing to misread, everything to cite.

Synthesis
Drawing many separate facts into one coherent account of a topic. Here it is grounded: built only from your verified facts, each claim cited, honest about the gaps.

Thread
One research conversation, saved. Keep one per topic so lines of inquiry don't tangle; threads persist between sessions.

Tier upgrade
Re-grounding an existing fact on a firmer source and raising its provenance to match, without changing the fact's wording. A guess becomes a cited fact over time.

Triangulation
Testing a claim against several independent sources at once and judging whether they agree. The verdict is not "a source says so" but "the sources concur."

Undisputed fact
A fact you marked as your own creative invention (`u`, ※) — true within your work by decree, with no external truth to check it against. It sits outside the trust ladder as an axiom.

AFTERWORD

About the author

Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is a discipline of **grounding** — of never reporting a number the system cannot back up, of knowing the provenance of every signal. It is not an accident that a tool he built for writers carries the same instinct into the library.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but

in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work.

Why the Research Assistant exists

Inkhaven is Vladimir's personal reflection on how a literary tool can help the people who write books. The Research Assistant is one answer to a specific frustration: that the tools writers use to **gather** facts and the tools they use to **write** live in different worlds, and the seam between them is where errors and lost citations breed.

A novelist keeps a browser open in one window and a manuscript in another, and the fact they looked up on Tuesday is gone by Thursday. A non-fiction author tracks citations in one program, prose in a second, and prays the two stay in sync. Neither tool remembers **where a fact came from** once it lands in the draft. The Research Assistant was built to close that gap: to make research a room inside the writing tool, where every fact you keep remembers its own provenance and can be composed straight back into the page.

A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don't.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an "Enterprise" tier you have to escape from.

This was a deliberate choice. There are excellent commercial tools for research and writing. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the graduate student writing a dissertation on a battered laptop, for the novelist who shouldn't have to pick between rent and software, for the engineer drafting in the same terminal where

they already write code, for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If Inkhaven helps you finish your book — that is enough. If it gives you a chord pattern you adapt into your own tool — that's a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

Where to find more

GitHub	@vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.
LinkedIn	/in/vladimirulogov — posts on observability, the occasional long-form essay.
YouTube	@vulogov — talks and walkthroughs from the conference trail.
X / Twitter	@vladimir_ulogov

If a fact you grounded ever surprises you, or a command trips you up and you can't find the answer in this book — open an issue on GitHub. The author reads them.

END OF THE BOOK · INKHAVEN 1.8.30